Module 1
Embedded system Design
BEC601

Introduction to Embedded System

What is Embedded System?

An Electronic/Electro mechanical system which is designed to perform a specific function and is a combination of both hardware and firmware (Software)

E.g. Electronic Toys, Mobile Handsets, Washing Machines, Air Conditioners, Automotive Control Units, Set Top Box, DVD Player etc…

Embedded Systems are:

- ❑ Unique in character and behavior
- ❑ With specialized hardware and software

Embedded Systems Vs General Computing Systems

General Purpose System	Embedded System
A system which is a combination of generic hardware and General Purpose Operating System for executing a variety of applications	A system which is a combination of special purpose hardware and embedded OS for executing a specific set of applications
Contain a General Purpose Operating System (GPOS)	May or may not contain an operating system for functioning
Applications are alterable (programmable) by user (It is possible for the end user to re-install the Operating System, and add or remove user applications)	The firmware of the embedded system is pre-programmed and it is non-alterable by end-user (There may be exceptions for systems supporting OS kernel image flashing through special hardware settings)
Performance is the key deciding factor on the selection of the system. Always 'Faster is Better'	Application specific requirements (like performance, power requirements, memory usage etc) are the key deciding factors
Less/not at all tailored towards reduced operating power requirements, options for different levels of power management.	Highly tailored to take advantage of the power saving modes supported by hardware and Operating System
Response requirements are not time critical	For certain category of embedded systems like mission critical systems, the response time requirement is highly critical
Need not be deterministic in execution behavior	Execution behavior is deterministic for certain type of embedded systems like 'Hard Real Time' systems

Introduction to Embedded System

History of Embedded Systems:

First Recognized Modern Embedded System: Apollo Guidance Computer (AGC)

First Mass Produced Embedded System: *Autonetics* ***D-17*** Guidance computer

Classification of Embedded Systems:

- ❑ Based on Generation
- ❑ Based on Complexity & Performance Requirements
- ❑ Based on deterministic behavior
- ❑ Based on Triggering

Introduction to Embedded System

Embedded Systems - Classification based on Generation

First Generation: The early embedded systems built around 8bit microprocessors like 8085 and Z80 and 4bit microcontrollers

Second Generation: Embedded Systems built around 16bit microprocessors and 8 or 16bit microcontrollers, following the first generation embedded systems

Third Generation: Embedded Systems built around high performance 16/32 bit Microprocessors/controllers, Application Specific Instruction set processors like Digital Signal Processors (DSPs), and Application Specific Integrated Circuits (ASICs)

Fourth Generation: Embedded Systems built around System on Chips (SoCs), Re-configurable processors and multicore processors

Introduction to Embedded System

Embedded Systems - Classification based on Complexity & Performance

Small Scale: simple in application. Performance requirements are not time critical. around 8/16 bit microprocessors/ microcontrollers. May or may not contain OS.

Medium Scale: Embedded Systems built around 16/32bit microprocessors/microcontrollers. Complex in hardware and software. Contain OS.

Large Scale/Complex: Embedded Systems built around high performance 32/64 bit Microprocessors/controllers, RTOS. SOC multiple processors and PLDs.

Introduction to Embedded System

Major Application Areas of Embedded Systems

- Consumer Electronics: Camcorders, Cameras etc.
- Household Appliances: Television, DVD players, Washing machine, Fridge, Microwave Oven etc.
- Home Automation and Security Systems: Air conditioners, sprinklers, Intruder detection alarms, Closed Circuit Television Cameras, Fire alarms etc.
- Automotive Industry: Anti-lock breaking systems (ABS), Engine Control, Ignition Systems, Automatic Navigation Systems etc.
- Telecom: Cellular Telephones, Telephone switches, Handset Multimedia Applications etc.

Introduction to Embedded System

Major Application Areas of Embedded Systems

- Computer Peripherals: Printers, Scanners, Fax machines etc.
- Computer Networking Systems: Network Routers, Switches, Hubs, Firewalls etc.
- Health Care: Different Kinds of Scanners, EEG, ECG Machines etc.
- Measurement & Instrumentation: Digital multi meters, Digital CROs, Logic Analyzers PLC systems etc.
- Banking & Retail: Automatic Teller Machines (ATM) and Currency counters, Point of Sales (POS)
- Card Readers: Barcode, Smart Card Readers, Hand held Devices etc.

Introduction to Embedded System

Purpose of Embedded Systems

Each Embedded Systems is designed to serve the purpose of any one or a combination of the following tasks.

- ❑ Data Collection/Storage/Representation
- ❑ Data Communication
- ❑ Data (Signal) Processing
- ❑ Monitoring
- ❑ Control
- ❑ Application Specific User Interface

Introduction to Embedded System

Purpose of Embedded Systems – Data Collection/Storage/Representation

✓Performs acquisition of data from the external world.

✓The collected data can be either analog or digital

✓Data collection is usually done for storage, analysis, manipulation and transmission

✓The collected data may be stored directly in the system or may be transmitted to some other systems or it may be processed by the system or it may be deleted instantly after giving a meaningful representation

Digital Camera for Image capturing/storage/display
Photo Courtesy of Casio -Model EXILIM ex-Z850
(www.casio.com)

Introduction to Embedded System

Purpose of Embedded Systems – Data Communication

- ✓ Embedded Data communication systems are deployed in applications ranging from complex satellite communication systems to simple home networking systems
- ✓ Embedded Data communication systems are dedicated for data communication
- ✓ The data communication can happen through a wired interface (like Ethernet, RS-232C/USB/IEEE1394 etc) or wireless interface (like Wi-Fi, GSM,/GPRS, Bluetooth, ZigBee etc)
- ✓ Network hubs, Routers, switches, Modems etc are typical examples for dedicated data transmission embedded systems

Wireless Network Router for Data Communication
Photo Courtesy of Linksys (www.linksys.com).
A division of CISCO system

Introduction to Embedded System

Purpose of Embedded Systems – Data (Signal) Processing

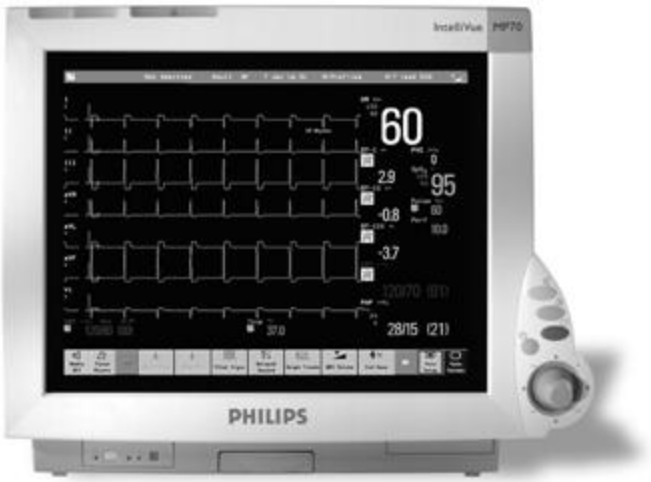
- ✓Embedded systems with Signal processing functionalities are employed in applications demanding signal processing like Speech coding, synthesis, audio video codec, transmission applications etc
- ✓Computational intensive systems
- ✓Employs Digital Signal Processors (DSPs)

Digital hearing Aid employing Signal Processing Technique
Siemens TRIANO 3 Digital hearing aid;
Siemens Audiology Copyright © 2005

Introduction to Embedded System

Purpose of Embedded Systems – Monitoring

- ✓ Embedded systems coming under this category are specifically designed for monitoring purpose
- ✓ They are used for determining the state of some variables using input sensors
- ✓ They cannot impose control over variables.
- ✓ Electro Cardiogram (ECG) machine for monitoring the heart beat of a patient is a typical example for this
- ✓ The sensors used in ECG are the different Electrodes connected to the patient's body
- ✓ Measuring instruments like Digital CRO, Digital Multi meter, Logic Analyzer etc used in Control & Instrumentation applications are also examples of embedded systems for monitoring purpose



Patient Monitoring system
Photo courtesy of Philips Medical Systems (www.medical.philips.com/)

Introduction to Embedded System

Purpose of Embedded Systems – Control

- ✓ Embedded systems with control functionalities are used for imposing control over some variables according to the changes in input variables
- ✓ Embedded system with control functionality contains both sensors and actuators
- ✓ Sensors are connected to the input port for capturing the changes in environmental variable or measuring variable
- ✓ The actuators connected to the output port are controlled according to the changes in input variable to put an impact on the controlling variable to bring the controlled variable to the specified range
- ✓ Air conditioner for controlling room temperature is a typical example for embedded system with 'Control' functionality
- ✓ Air conditioner contains a room temperature sensing element (sensor) which may be a thermistor and a handheld unit for setting up (feeding) the desired temperature
- ✓ The air compressor unit acts as the actuator. The compressor is controlled according to the current room temperature and the desired temperature set by the end user.

Air Conditioner for controlling room temperature
Photo Courtesy of Electrolux Corporation
(www.electrolux.com/au)

Introduction to Embedded System

Purpose of Embedded Systems – Application Specific User Interface

- ✓Embedded systems which are designed for a specific application
- ✓Contains Application Specific User interface (rather than general standard UI) like key board, Display units etc
- ✓Aimed at a specific target group of users
- ✓Mobile handsets, Control units in industrial applications etc are examples for this

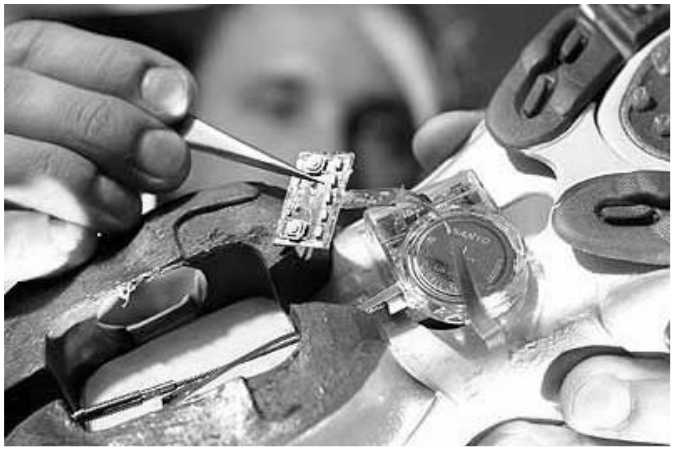
Patient Monitoring system
Photo courtesy of Philips Medical Systems
(www.medical.philips.com/)

Introduction to Embedded System

'Smart' running shoes from Adidas – The Innovative bonding of Life Style with Embedded Technology

- ✓ Shoe developed by Adidas, which constantly adapts its shock-absorbing characteristics to customize its value to the individual runner, depending on running style, pace, body weight, and running surface
- ✓ It contains sensors, actuators and a microprocessor unit which runs the algorithm for adapting the shock-absorbing characteristics of the shoe
- ✓ A 'Hall effect sensor' placed at the top of the "cushioning element" senses the compression and passes it to the Microprocessor
- ✓ A micro motor actuator controls the cushioning as per the commands from the MPU, based on the compression sensed by the 'Hall effect sensor'

What an innovative bonding of Embedded Technology with Real life needs !!!☺

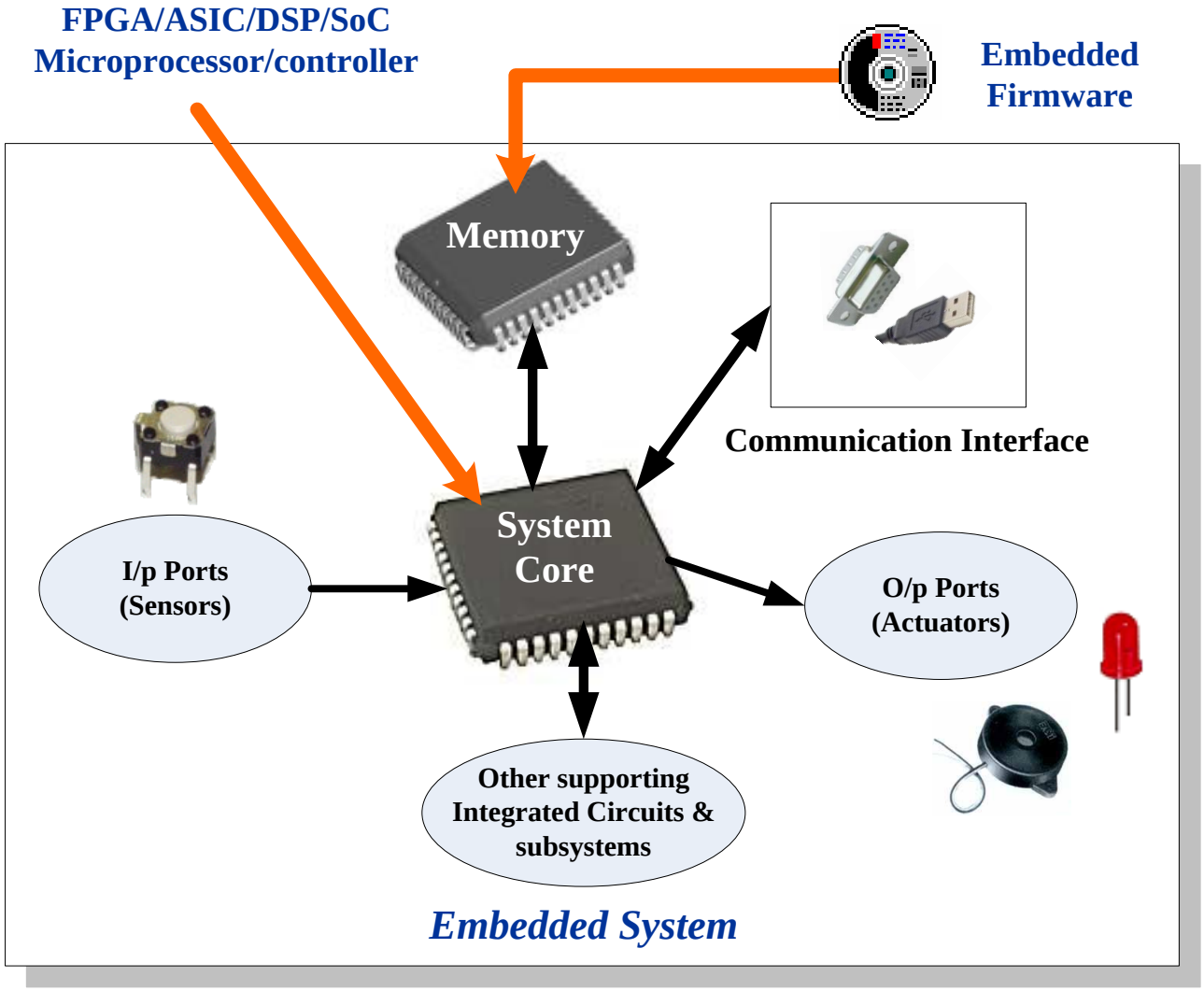


Electronics-enabled "Smart" running shoes from Adidas
Photo Courtesy of Adidas – Salomon AG (www.adidas.com)

Typical Embedded System consists of

- Core of the Embedded System,
- Memory,
- Sensors and Actuators,
- Communication Interface,
- Embedded Firmware,
- Other System Components

The Typical Embedded System



The Typical Embedded System

The Core of the Embedded Systems

The core of the embedded system falls into any one of the following categories.

- **General Purpose and Domain Specific Processors**
  - Microprocessors
  - Microcontrollers
  - Digital Signal Processors
- **Programmable Logic Devices (PLDs)**
- **Application Specific Integrated Circuits (ASICs)**
- **Commercial off the shelf Components (COTS)**

The Typical Embedded System

General Purpose Processor (GPP) Vs Application Specific Instruction Set Processor (ASIP)

✓General Purpose Processor or GPP is a processor designed for general computational tasks

✓GPPs are produced in large volumes and targeting the general market. Due to the high volume production, the per unit cost for a chip is low compared to ASIC or other specific ICs

✓A typical general purpose processor contains an Arithmetic and Logic Unit (ALU) and Control Unit (CU)

✓Application Specific Instruction Set processors (ASIPs) are processors with architecture and instruction set optimized to specific domain/application requirements like Network processing, Automotive, Telecom, media applications, digital signal processing, control applications etc.

The Typical Embedded System

General Purpose Processor (GPP) Vs Application Specific Instruction Set Processor (ASIP)

✓ASIPs fill the architectural spectrum between General Purpose Processors and Application Specific Integrated Circuits (ASICs)

✓The need for an ASIP arises when the traditional general purpose processor are unable to meet the increasing application needs

✓Some Microcontrollers (like Automotive AVR, USB AVR from Atmel), System on Chips, Digital Signal Processors etc are examples of Application Specific Instruction Set Processors (ASIPs)

✓ASIPs incorporate a processor and on-chip peripherals, demanded by the application requirement, program and data memory

The Typical Embedded System

Microprocessor

✓A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions, which is specific to the manufacturer

✓In general the CPU contains the Arithmetic and Logic Unit (ALU), Control Unit and Working registers

✓Microprocessor is a dependant unit and it requires the combination of other hardware like Memory, Timer Unit, and Interrupt Controller etc for proper functioning.

✓Intel claims the credit for developing the first Microprocessor unit Intel 4004, a 4 bit processor which was released in Nov 1971

The Typical Embedded System

Microcontroller

- ✓ A highly integrated silicon chip containing a CPU, scratch pad RAM, Special and General purpose Register Arrays, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports
- ✓ Microcontrollers can be considered as a super set of Microprocessors
- ✓ Microcontroller can be general purpose (like Intel 8051, designed for generic applications and domains) or application specific (Like Automotive AVR from Atmel Corporation. Designed specifically for automotive applications)
- ✓ Since a microcontroller contains all the necessary functional blocks for independent working, they found greater place in the embedded domain in place of microprocessors
- ✓ Microcontrollers are cheap, cost effective and are readily available in the market
- ✓ Texas Instruments TMS 1000 is considered as the world's first microcontroller

The Typical Embedded System

Microprocessor Vs Microcontroller

Microprocessor	Microcontroller
A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions	A microcontroller is a highly integrated chip that contains a CPU, scratch pad RAM, Special and General purpose Register Arrays, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports
It is a dependent unit. It requires the combination of other chips like Timers, Program and data memory chips, Interrupt controllers etc for functioning	It is a self contained unit and it doesn't require external Interrupt Controller, Timer, UART etc for its functioning
Most of the time general purpose in design and operation	Mostly application oriented or domain specific
Doesn't contain a built in I/O port. The I/O Port functionality needs to be implemented with the help of external Programmable Peripheral Interface Chips like 8255	Most of the processors contain multiple built-in I/O ports which can be operated as a single 8 or 16 or 32 bit Port or as individual port pins
Targeted for high end market where performance is important	Targeted for embedded market where performance is not so critical (At present this demarcation is invalid)
Limited power saving options compared to microcontrollers	Includes lot of power saving features

The Typical Embedded System

Digital Signal Processors (DSPs)

Powerful special purpose 8/16/32 bit microprocessors designed specifically to meet the computational demands and power constraints of today's embedded audio, video, and communications applications

✓Digital Signal Processors are 2 to 3 times faster than the general purpose microprocessors in signal processing applications

✓DSPs implement algorithms in hardware which speeds up the execution whereas general purpose processors implement the algorithm in firmware and the speed of execution depends primarily on the clock for the processors

The Typical Embedded System

Digital Signal Processors (DSPs)

✓DSP can be viewed as a microchip designed for performing high speed computational operations for ‘addition’, ‘subtraction’, ‘multiplication’ and ‘division’

✓A typical Digital Signal Processor incorporates the following key units

- ✓Program Memory
- ✓Data Memory
- ✓Computational Engine
- ✓I/O Unit

✓Audio video signal processing, telecommunication and multimedia applications are typical examples where DSP is employed

The Typical Embedded System

RISC V/s CISC Processors/Controllers

RISC	CISC
Lesser no. of instructions	Greater no. of Instructions
Instruction Pipelining and increased execution speed	Generally no instruction pipelining feature
Orthogonal Instruction Set (Allows each instruction to operate on any register and use any addressing mode)	Non Orthogonal Instruction Set (All instructions are not allowed to operate on any register and use any addressing mode. It is instruction specific)
Operations are performed on registers only, the only memory operations are load and store	Operations are performed on registers or memory depending on the instruction
Large number of registers are available	Limited no. of general purpose registers
Programmer needs to write more code to execute a task since the instructions are simpler ones	Instructions are like macros in C language. A programmer can achieve the desired functionality with a single instruction which in turn provides the effect of using more simpler single instructions in RISC
Single, Fixed length Instructions	Variable length Instructions
Less Silicon usage and pin count	More silicon usage since more additional decoder logic is required to implement the complex instruction decoding.
With Harvard Architecture	Can be Harvard or Von-Neumann Architecture

The Typical Embedded System

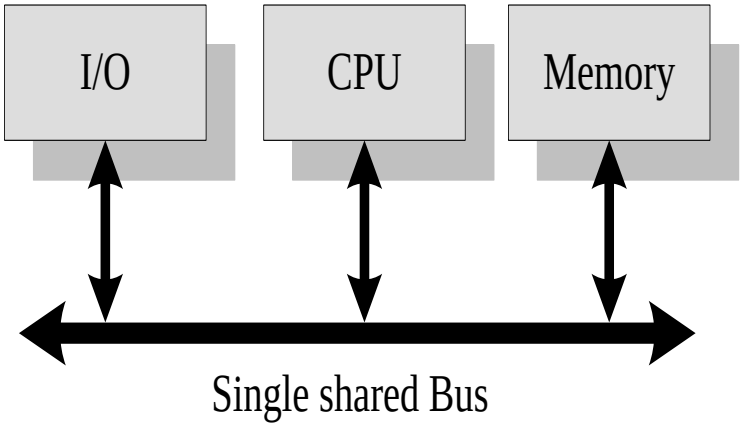
Harvard V/s Von-Neumann Processor/Controller Architecture

- ✓ The terms Harvard and Von-Neumann refers to the processor architecture design.
- ✓ Microprocessors/controllers based on the **Von-Neumann** architecture shares a single common bus for fetching both instructions and data. Program instructions and data are stored in a common main memory
- ✓ Microprocessors/controllers based on the **Harvard** architecture will have separate data bus and instruction bus. This allows the data transfer and program fetching to occur simultaneously on both buses

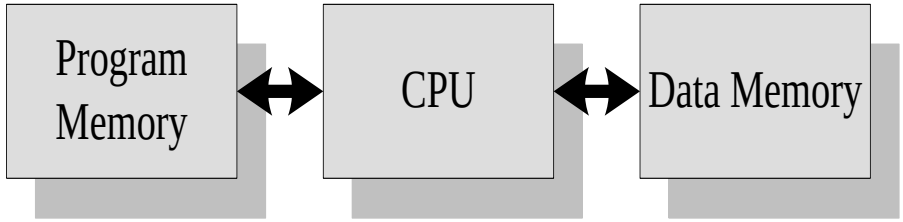
The Typical Embedded System

Harvard V/s Von-Neumann Processor/Controller Architecture

- ✓ With Harvard architecture, the data memory can be read and written while the program memory is being accessed. These separated data memory and code memory buses allow one instruction to execute while the next instruction is fetched ("Pre-fetching")



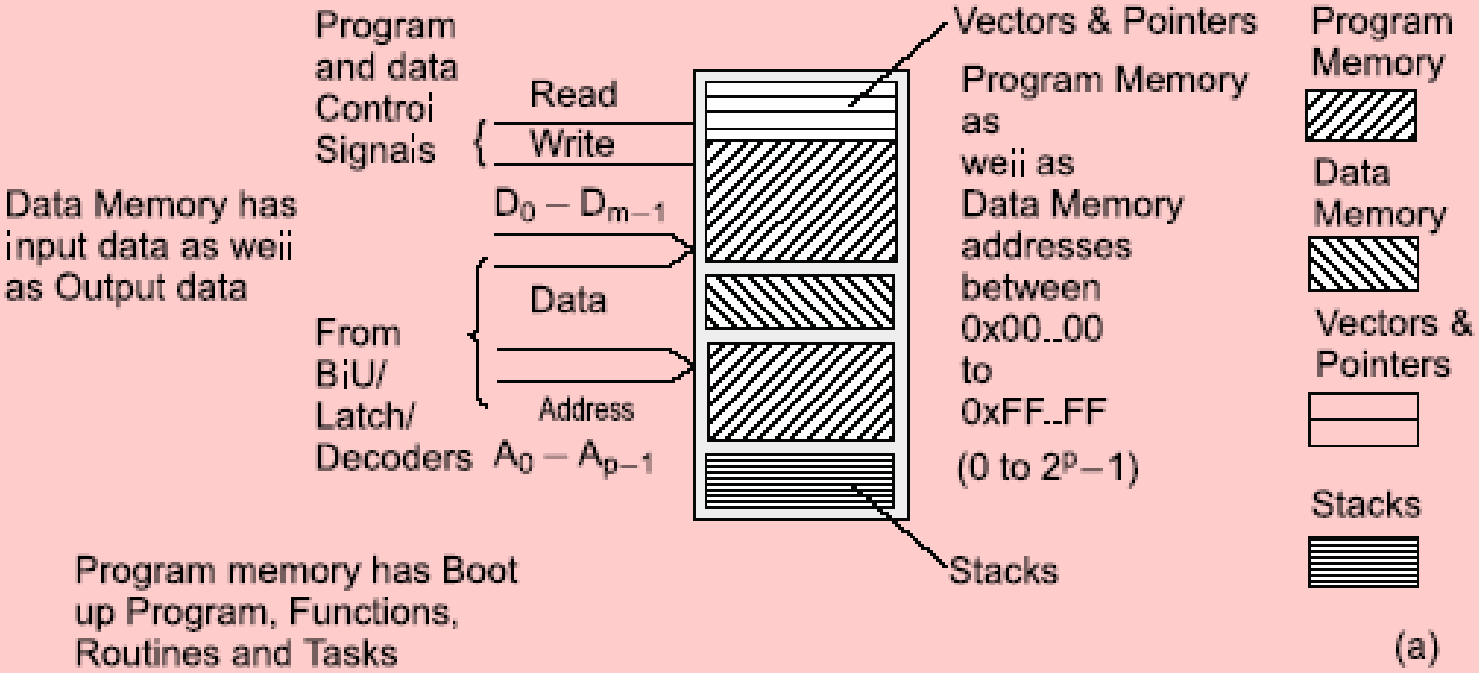
Von-Neumann Architecture



Harvard Architecture

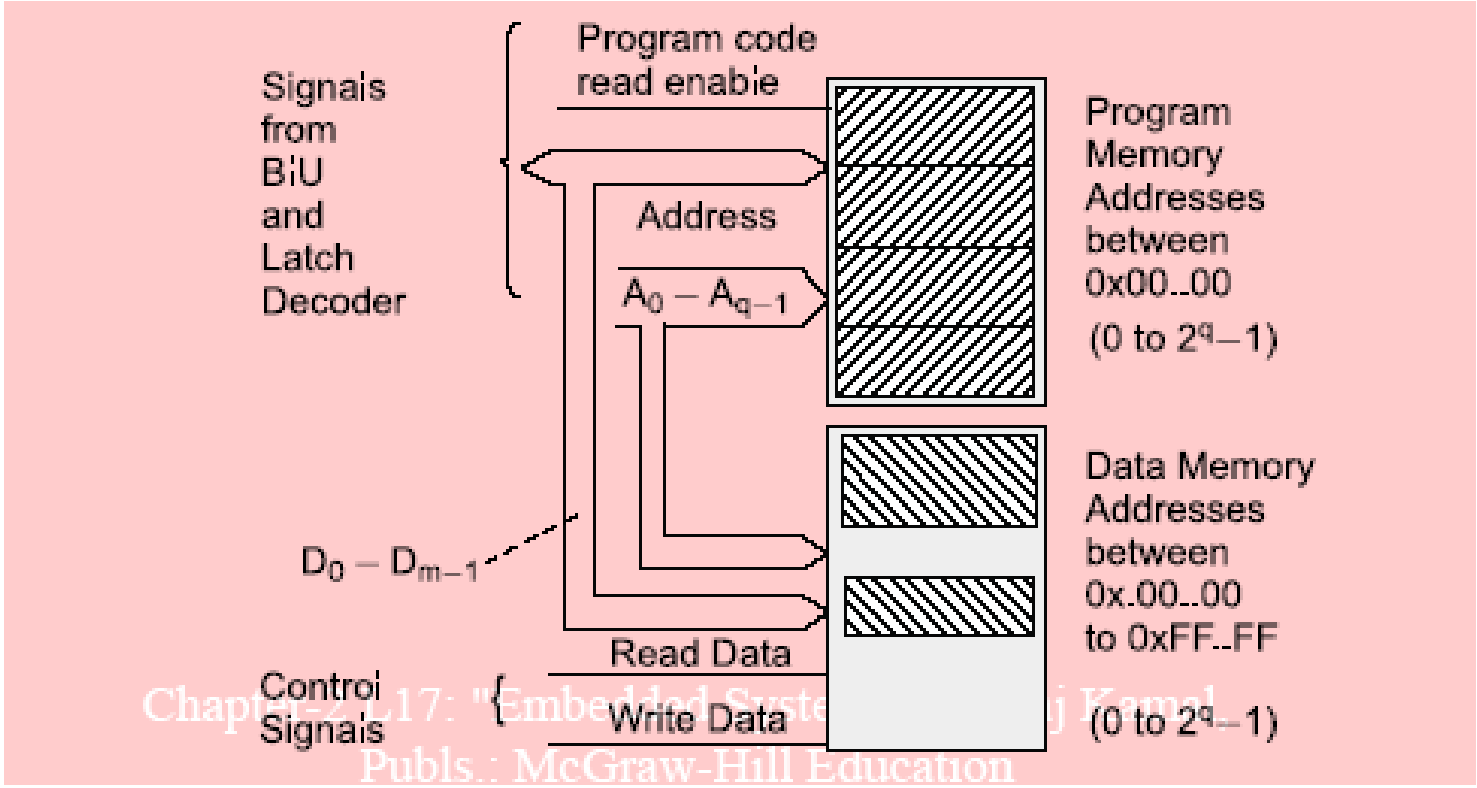
Princeton Architecture

- 80x86 processors and ARM7 have Princeton architecture for main memory.
- Vectors and pointers, variables, program segments and memory blocks for data and stacks have different addresses in the program in Princeton memory architecture



Harvard architecture

- 8051-family microcontrollers have Harvard architecture
- Separate data buses ensure simultaneous accesses for instructions and data
- When the address spaces for the data and for program are distinct



The Typical Embedded System

Harvard V/s Von-Neumann Processor/Controller Architecture

Harvard Architecture	Von-Neumann Architecture
Separate buses for Instruction and Data fetching	Single shared bus for Instruction and Data fetching
Easier to Pipeline, so high performance can be achieved	Low performance Compared to Harvard Architecture
Comparatively high cost	Cheaper
No memory alignment problems	Allows self modifying codes
Since data memory and program memory are stored physically in different locations, no chances for accidental corruption of program memory	Since data memory and program memory are stored physically in same chip, chances for accidental corruption of program memory

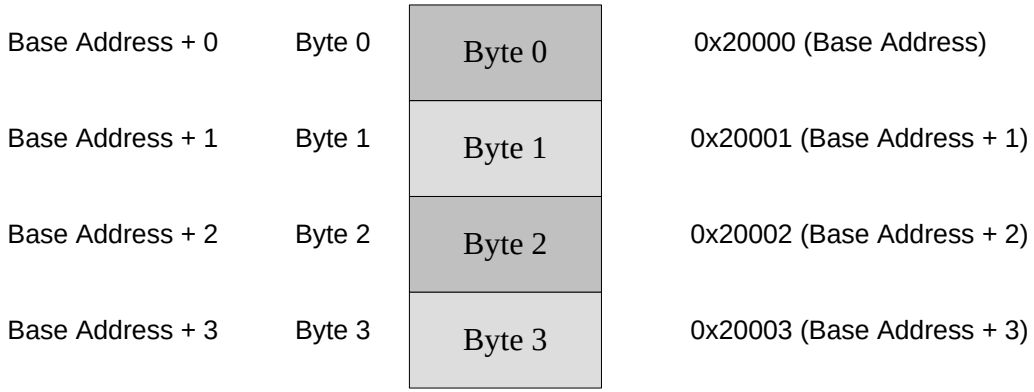
The Typical Embedded System

Big-endian V/s Little-endian processors

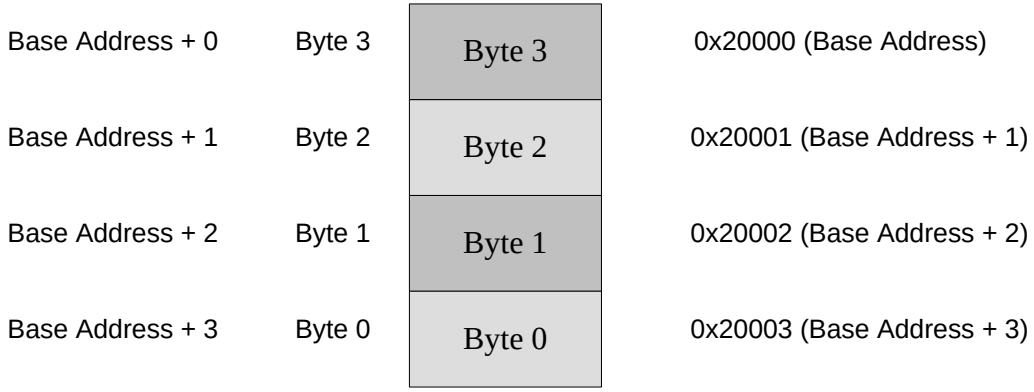
- ✓ Endianness specifies the order in which the data is stored in the memory by processor operations in a multi byte system (Processors whose word size is greater than one byte). Suppose the word length is two byte then data can be stored in memory in two different ways
  - ✓ Higher order of data byte at the higher memory and lower order of data byte at location just below the higher memory
  - ✓ Lower order of data byte at the higher memory and higher order of data byte at location just below the higher memory
- ✓ ***Little-endian*** means the lower-order byte of the data is stored in memory at the lowest address, and the higher-order byte at the highest address. (The little end comes first)
- ✓ ***Big-endian*** means the higher-order byte of the data is stored in memory at the lowest address, and the lower-order byte at the highest address. (The big end comes first.)

The Typical Embedded System

Big-endian V/s Little-endian processors



Little-endian Operation

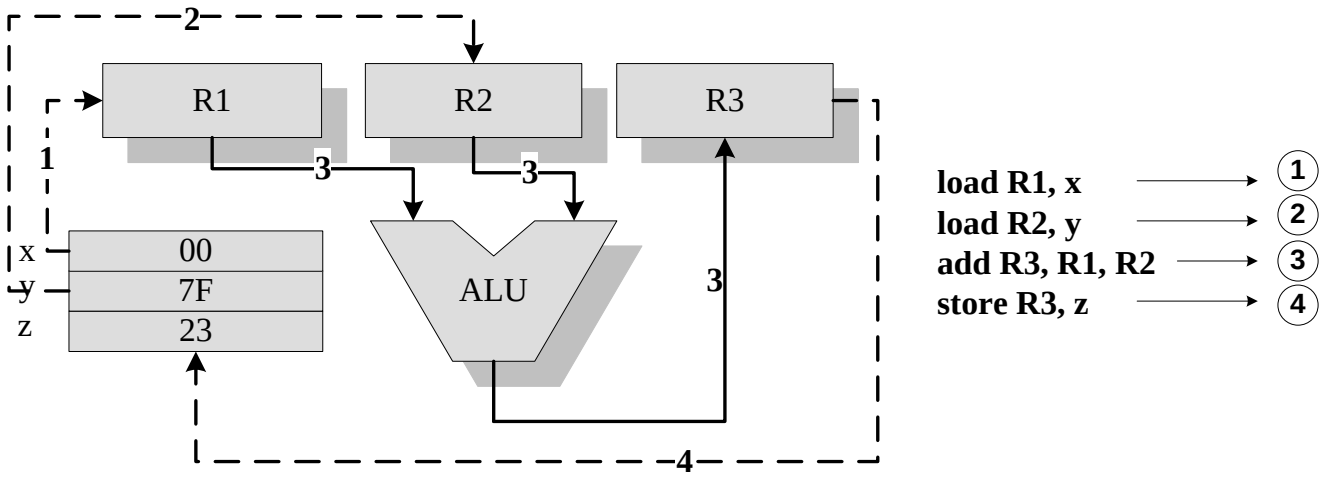


Big-endian Operation

The Typical Embedded System

Load Store Operation & Instruction Pipelining

The RISC processor instruction set is orthogonal and it operates on registers. The memory access related operations are performed by the special instructions *load* and *store*. If the operand is specified as memory location, the content of it is loaded to a register using the *load* instruction. The instruction *store* stores data from a specified register to a specified memory location

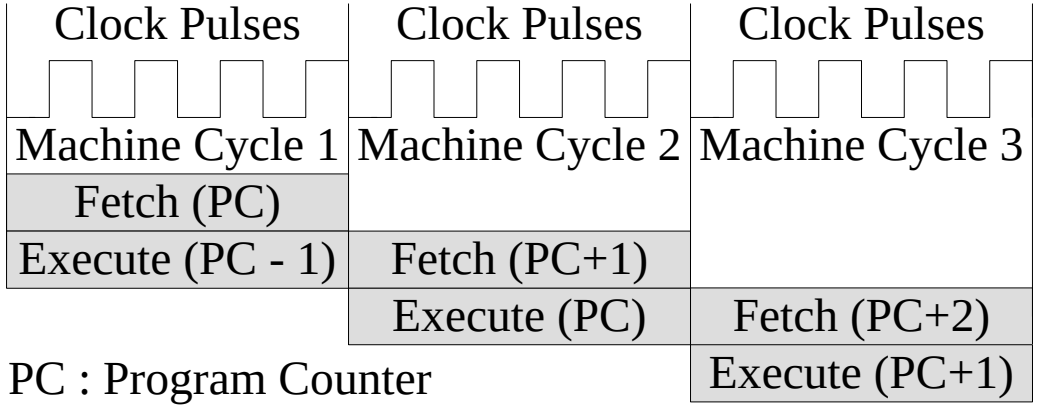


Load Store Operation

The Typical Embedded System

Instruction Pipelining

- ✓ The conventional instruction execution by the processor follows the fetch-decode-execute sequence
- ✓ The 'fetch' part fetches the instruction from program memory or code memory and the decode part decodes the instruction to generate the necessary control signals
- ✓ The execute stage reads the operands, perform ALU operations and stores the result. In conventional program execution, the fetch and decode operations are performed in sequence
- ✓ During the decode operation the memory address bus is available and if it possible to effectively utilize it for an instruction fetch, the processing speed can be increased
- ✓ In its simplest form instruction pipelining refers to the overlapped execution of instructions



The Single stage pipelining concept

The Typical Embedded System

Programmable Logic Devices (PLDs)

- ✓ Logic devices provide specific functions, including device-to-device interfacing, data communication, signal processing, data display, timing and control operations, and almost every other function a system must perform.
- ✓ Logic devices can be classified into two broad categories - Fixed and Programmable. The circuits in a fixed logic device are permanent, they perform one function or set of functions - once manufactured, they cannot be changed
- ✓ Programmable logic devices (PLDs) offer customers a wide range of logic capacity, features, speed, and voltage characteristics - and these devices can be re-configured to perform any number of functions at any time
- ✓ Designers can use inexpensive software tools to quickly develop, simulate, and test their logic designs in PLD based design. The design can be quickly programmed into a device, and immediately tested in a live circuit
- ✓ PLDs are based on re-writable memory technology and the device is reprogrammed to change the design

The Typical Embedded System

Programmable Logic Devices (PLDs) – CPLDs and FPGA

- ✓ Field Programmable Gate Arrays (FPGAs) and Complex Programmable Logic Devices (CPLDs) are the two major types of programmable logic devices
- ✓ FPGAs offer the highest amount of logic density, the most features, and the highest performance.
- ✓ These advanced FPGA devices also offer features such as built-in hardwired processors (such as the IBM Power PC), substantial amounts of memory, clock management systems, and support for many of the latest, very fast device-to-device signaling technologies
- ✓ **FPGAs are used in a wide variety of applications ranging from data processing and storage, to instrumentation, telecommunications, and digital signal processing**
- ✓ CPLDs, by contrast, offer much smaller amounts of logic - up to about 10,000 gates
- ✓ **CPLDs offer very predictable timing characteristics and are therefore ideal for critical control applications**
- ✓ CPLDs such as the Xilinx **CoolRunner** series also require extremely low amounts of power and are very inexpensive, making them ideal for cost-sensitive, battery-operated, portable applications such as mobile phones and digital handheld assistants.

The Typical Embedded System

Application Specific Integrated Circuit (ASIC)

- ✓ A microchip designed to perform a specific or unique application. It is used as replacement to conventional general purpose logic chips.
- ✓ ASIC integrates several functions into a single chip and thereby reduces the system development cost
- ✓ Most of the ASICs are proprietary products. As a single chip, ASIC consumes very small area in the total system and thereby helps in the design of smaller systems with high capabilities/functionalities.
- ✓ ASICs can be pre-fabricated for a special application or it can be custom fabricated by using the components from a re-usable '*building block*' library of components for a particular customer application
- ✓ Fabrication of ASICs requires a non-refundable initial investment (Non Recurring Engineering (NRE) charges) for the process technology and configuration expenses
- ✓ If the Non-Recurring Engineering Charges (NRE) is born by a third party and the Application Specific Integrated Circuit (ASIC) is made openly available in the market, the ASIC is referred as Application Specific Standard Product (ASSP)

The Typical Embedded System

Commercial off the Shelf Component (COTS)

- ✓ A Commercial off-the-shelf (COTS) product is one which is used '*as-is*'
- ✓ COTS products are designed in such a way to provide easy integration and interoperability with existing system components
- ✓ Typical examples for the COTS hardware unit are Remote Controlled Toy Car control unit including the RF Circuitry part, High performance, high frequency microwave electronics (2 to 200 GHz), High bandwidth analog-to-digital converters, Devices and components for operation at very high temperatures, Electro-optic IR imaging arrays, UV/IR Detectors etc
- ✓ A COTS component in turn contains a General Purpose Processor (GPP) or Application Specific Instruction Set Processor (ASIP) or Application Specific Integrated Chip (ASIC)/Application Specific Standard Product (ASSP) or Programmable Logic Device (PLD)
- ✓ The major advantage of using COTS is that they are readily available in the market, cheap and a developer can cut down his/her development time to a great extend

The Typical Embedded System

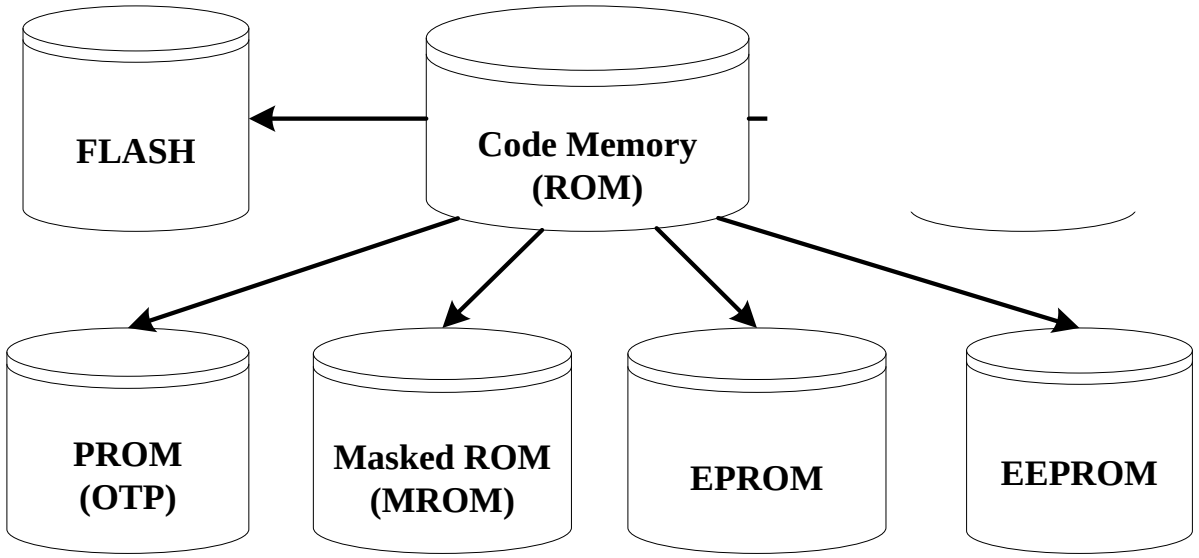
Memory

- ✓ Memory is an important part of an embedded system. The memory used in embedded system can be either Program Storage Memory (ROM) or Data memory (RAM)
- ✓ Certain Embedded processors/controllers contain built in program memory and data memory and this memory is known as on-chip memory

The Typical Embedded System

Memory – Program Storage Memory

- ✓ Stores the program instructions
- ✓ Retains its contents even after the power to it is turned off. It is generally known as Non volatile storage memory
- ✓ Depending on the fabrication, erasing and programming techniques they are classified into



The Typical Embedded System

Memory – Program Storage Memory – Masked ROM (MROM)

- ✓ One-time programmable memory. Uses hardwired technology for storing data. The device is factory programmed by masking and metallization process according to the data provided by the end user
- ✓ The primary advantage of MROM is low cost for high volume production. They are the least expensive type of solid state memory
- ✓ Different mechanisms are used for the masking process of the ROM, like
  - ✓ Creation of an enhancement or depletion mode transistor through channel implant
  - ✓ By creating the memory cell either using a standard transistor or a high threshold transistor. In the high threshold mode, the supply voltage required to turn ON the transistor is above the normal ROM IC operating voltage. This ensures that the transistor is always off and the memory cell stores always logic 0.
- ✓ The limitation with MROM based firmware storage is the inability to modify the device firmware against firmware upgrades. Since the MROM is permanent in bit storage, it is not possible to alter the bit information

The Typical Embedded System

Memory – Program Storage Memory – Programmable Read Only Memory (PROM) / (OTP)

- ✓ Unlike MROM it is not pre-programmed by the manufacturer
- ✓ PROM/OTP has *nichrome* or *polysilicon* wires arranged in a matrix, these wires can be functionally viewed as fuses
- ✓ It is programmed by a PROM programmer which selectively burns the fuses according to the bit pattern to be stored
- ✓ Fuses which are not blown/burned represents a logic “1” where as fuses which are blown/burned represents a logic “0”.The default state is logic “1”
- ✓ OTP is widely used for commercial production of embedded systems whose proto-typed versions are proven and the code is finalized
- ✓ It is a low cost solution for commercial production. OTPs cannot be reprogrammed

The Typical Embedded System

Memory – Program Storage Memory – Erasable Programmable Read Only Memory (EPROM)

- ✓ Erasable Programmable Read Only (EPROM) memory gives the flexibility to re-program the same chip
- ✓ EPROM stores the bit information by charging the floating gate of an FET
- ✓ Bit information is stored by using an EPROM Programmer, which applies high voltage to charge the floating gate
- ✓ EPROM contains a quartz crystal window for erasing the stored information. If the window is exposed to Ultra violet rays for a fixed duration, the entire memory will be erased
- ✓ Even though the EPROM chip is flexible in terms of re-programmability, it needs to be taken out of the circuit board and needs to be put in a UV eraser device for 20 to 30 minutes

The Typical Embedded System

Memory – Program Storage Memory – Electrically Erasable Programmable Read Only Memory (EEPROM)

- ✓ Erasable Programmable Read Only (EPROM) memory gives the flexibility to re-program the same chip using electrical signals
- ✓ The information contained in the EEPROM memory can be altered by using electrical signals at the register/Byte level
- ✓ They can be erased and reprogrammed within the circuit
- ✓ These chips include a chip erase mode and in this mode they can be erased in a few milliseconds
- ✓ It provides greater flexibility for system design
- ✓ The only limitation is their capacity is limited when compared with the standard ROM (A few kilobytes).

The Typical Embedded System

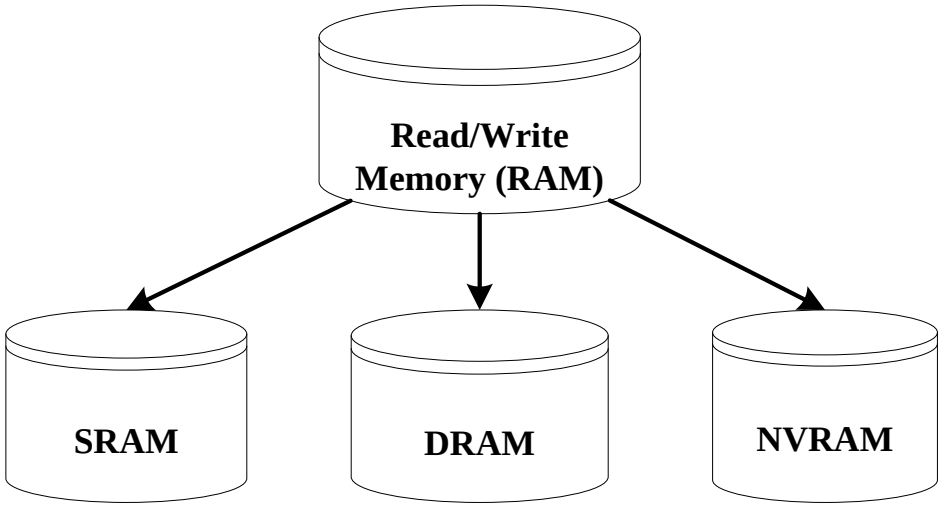
Memory – Program Storage Memory – FLASH

- ✓ FLASH memory is a variation of EEPROM technology
- ✓ It combines the re-programmability of EEPROM and the high capacity of standard ROMs
- ✓ FLASH memory is organized as sectors (blocks) or pages
- ✓ FLASH memory stores information in an array of floating gate MOSFET transistors
- ✓ The erasing of memory can be done at sector level or page level without affecting the other sectors or pages
- ✓ Each sector/page should be erased before re-programming

The Typical Embedded System

Memory – Read-Write Memory/Random Access Memory (RAM)

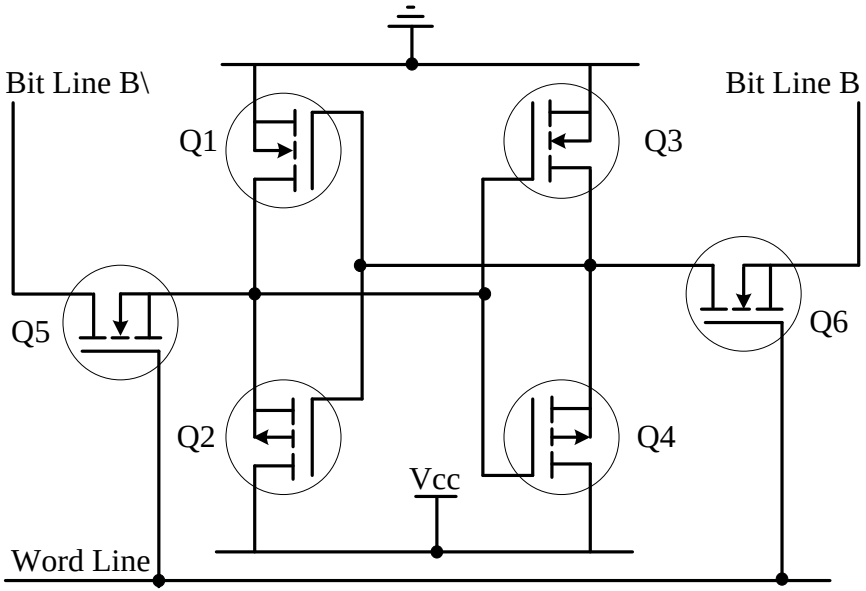
- ✓ RAM is the data memory or working memory of the controller/processor
- ✓ RAM is volatile, meaning when the power is turned off, all the contents are destroyed
- ✓ RAM is a direct access memory, meaning we can access the desired memory location directly without the need for traversing through the entire memory locations to reach the desired memory position (i.e. Random Access of memory location)



The Typical Embedded System

Memory – RAM – Static RAM (SRAM)

- ✓ Static RAM stores data in the form of Voltage. They are made up of flip-flops
- ✓ In typical implementation, an SRAM cell (bit) is realized using 6 transistors (or 6 MOSFETs). Four of the transistors are used for building the latch (flip-flop) part of the memory cell and 2 for controlling the access.
- ✓ Static RAM is the fastest form of RAM available. SRAM is fast in operation due to its resistive networking and switching capabilities

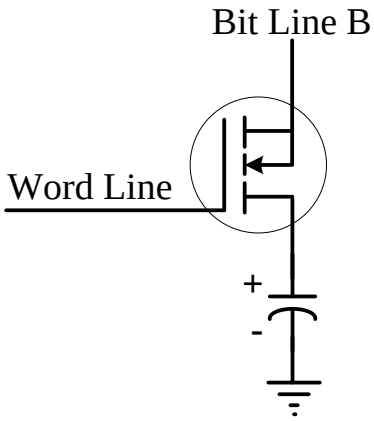


SRAM cell implementation

The Typical Embedded System

Memory – RAM – Dynamic RAM (DRAM)

- ✓ Dynamic RAM stores data in the form of charge. They are made up of MOS transistor gates
- ✓ The advantages of DRAM are its high density and low cost compared to SRAM
- ✓ The disadvantage is that since the information is stored as charge it gets leaked off with time and to prevent this they need to be refreshed periodically
- ✓ Special circuits called DRAM controllers are used for the refreshing operation. The refresh operation is done periodically in milliseconds interval



DRAM cell implementation

The Typical Embedded System

Memory – RAM – SRAM Vs DRAM)

SRAM Cell	DRAM Cell
Made up of 6 CMOS transistors (MOSFET)	Made up of a MOSFET and a capacitor
Doesn't Require refreshing	Requires refreshing
Low capacity (Less dense)	High Capacity (Highly dense)
More expensive	Less Expensive
Fast in operation. Typical access time is 10ns	Slow in operation due to refresh requirements. Typical access time is 60ns. Write operation is faster than read operation.

The Typical Embedded System

Memory – RAM – Non Volatile RAM (NVRAM)

- ✓ Random access memory with battery backup
- ✓ It contains Static RAM based memory and a minute battery for providing supply to the memory in the absence of external power supply
- ✓ The memory and battery are packed together in a single package
- ✓ NVRAM is used for the non volatile storage of results of operations or for setting up of flags etc
- ✓ The life span of NVRAM is expected to be around 10 years
- ✓ DS1744 from Maxim/Dallas is an example for 32KB NVRAM

Memory Selection

Memory Selection

ROM image file in memory

Once Software designer coding is over and the ROM image file is ready, a hardware designer of a system is faced with questions, of what type of memory and

what to use, how much size of each, should be used.

First step is to build a design table

- Decide the application type like
  - Automatic Washing machine
  - Data Acquisition Systems for the sixteen parameters channels
  - Data Acquisition Systems for the ECG waveforms
  - Multi channel Fast Encryption and cum decryption Transceiver System
  - Mobile Phone system

Memory Selection

- Processor used
  - Type
    - RISC/CISC
    - MicroProcessor, MicroControllers, DSP
    - Fixed Point, Floating Point
  - Example
    - RISC: Microchip, ARM Series(ARM7/9), SPARC, PowerPC
    - CISC: 8085, MCS31, MCS251,TMS320Cxx
    - MicroProcessor: 8086, 68K Series, Pentium, AMD
    - MicroController: MCS31, MCS251, TMS320C6x, Microchip
    - DSP: TI (TMS320C3x, 5x & 6x), ADSP (Shark, Tiger Shark, Hammer Head, Black Fin) , Oak, Teak, Pine, Palm …
    - Fixed Point: 8085, 68K, MCS251, TMS320C6x, Microchip
    - Floating Point: TI (TMS320C3x, TMS320C67x)
- Internal Memory (size available or needed)
  - ROM or PROM , EEPROM, RAM

Memory Selection

- External device used or needed (size)
  - ROM or EPROM , EEPROM or Flash device
  - RAM device
  - Parameterised distributed RAM
  - Parameterised Block RAM
- Software design has to define ROM, RAM allocations for various segments, data sets and structures
- However prior estimate of the memory type and size requirements can be made.

  Example, a system of 92 kB of memory is needed then a device of 128 kB is selected. Memory are available as 1k,2k,4k……..64k,128k,256k…

  (interms of 2^10+n, n=0,1,2……..)

The Typical Embedded System

Sensors & Actuators

Sensor:

A transducer device which converts energy from one form to another for any measurement or control purpose. Sensors acts as input device

Eg. Hall Effect Sensor which measures the distance between the cushion and magnet in the Smart Running shoes from adidas

The Typical Embedded System

Sensors & Actuators

Actuator:

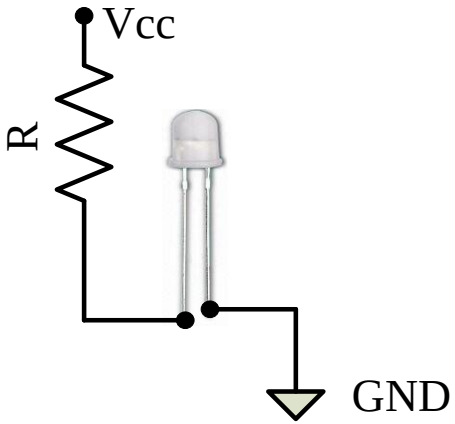
A form of transducer device (mechanical or electrical) which converts signals to corresponding physical action (motion). Actuator acts as an output device

Eg. Micro motor actuator which adjusts the position of the cushioning element in the Smart Running shoes from adidas

The Typical Embedded System

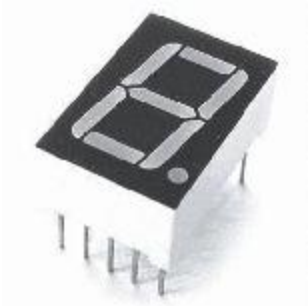
The I/O Subsystem – I/O Devices - Light Emitting Diode (LED)

- ✓ Light Emitting Diode (LED) is an output device for visual indication in any embedded system
- ✓ LED can be used as an indicator for the status of various signals or situations. Typical examples are indicating the presence of power conditions like 'Device ON', 'Battery low' or 'Charging of battery' for a battery operated handheld embedded devices
- ✓ LED is a p-n junction diode and it contains an anode and a cathode. For proper functioning of the LED, the anode of it should be connected to +ve terminal of the supply voltage and cathode to the –ve terminal of supply voltage
- ✓ The current flowing through the LED must limited to a value below the maximum current that it can conduct. A resister is used in series between the power supply and the resistor to limit the current through the LED

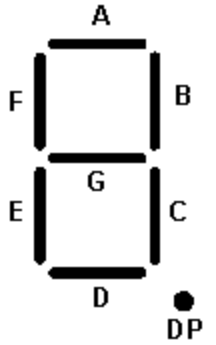


The Typical Embedded System

The I/O Subsystem – I/O Devices – 7-Segment LED Display



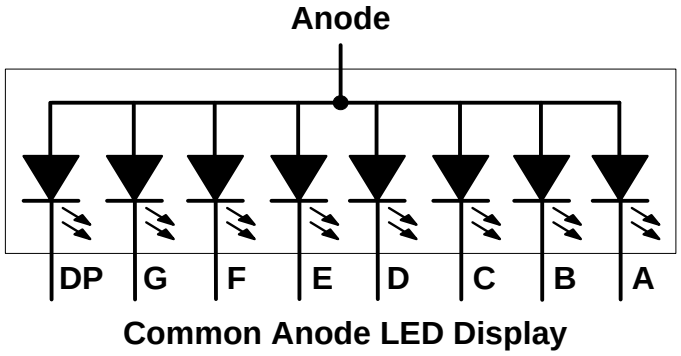
- ✓ The 7 – segment LED display is an output device for displaying alpha numeric characters
- ✓ It contains 8 light-emitting diode (LED) segments arranged in a special form. Out of the 8 LED segments, 7 are used for displaying alpha numeric characters
- ✓ The LED segments are named A to G and the decimal point LED segment is named as DP
- ✓ The LED Segments A to G and DP should be lit accordingly to display numbers and characters
- ✓ The 7 – segment LED displays are available in two different configurations, namely; Common anode and Common cathode
- ✓ In the Common anode configuration, the anodes of the 8 segments are connected commonly whereas in the Common cathode configuration, the 8 LED segments share a common cathode line

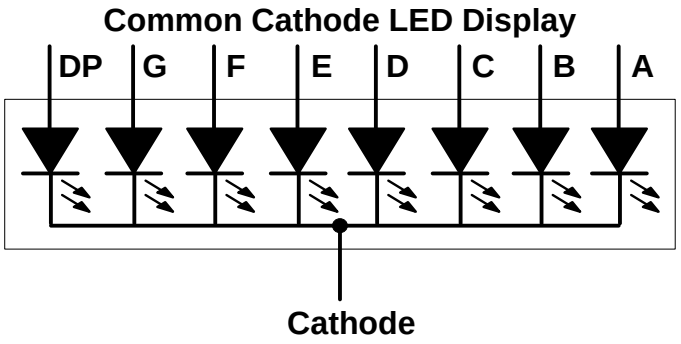


The Typical Embedded System

The I/O Subsystem – I/O Devices – 7-Segment LED Display

- ✓ Based on the configuration of the 7 – segment LED unit, the LED segment anode or cathode is connected to the Port of the processor/controller in the order 'A' segment to the Least significant port Pin and DP segment to the most significant Port Pin.
- ✓ The current flow through each of the LED segments should be limited to the maximum value supported by the LED display unit
- ✓ The typical value for the current falls within the range of 20mA
- ✓ The current through each segment can be limited by connecting a current limiting resistor to the anode or cathode of each segment

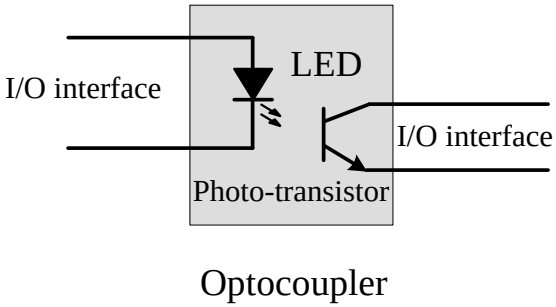




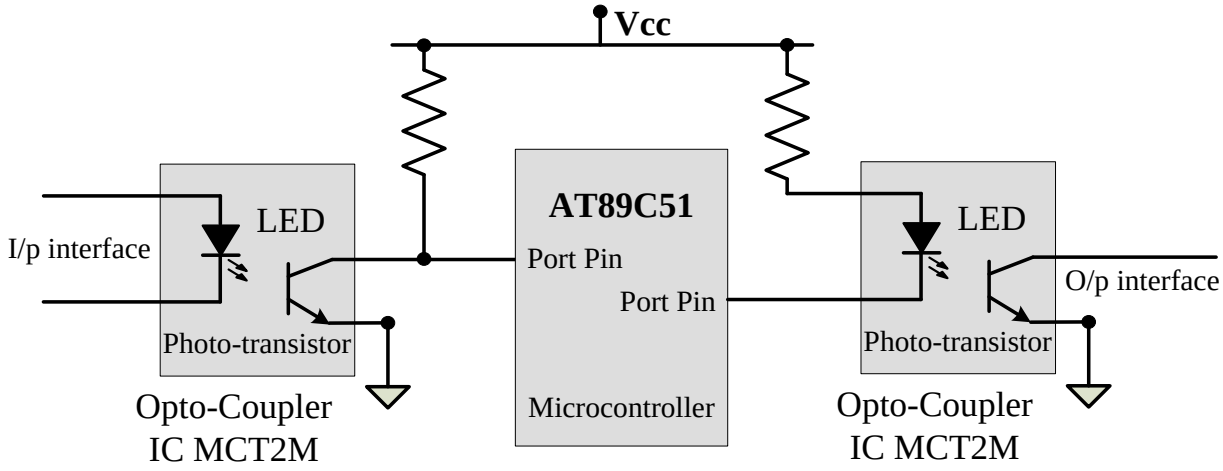
The Typical Embedded System

The I/O Subsystem – I/O Devices – Optocoupler

- ✓ Optocoupler is a solid state device to isolate two parts of a circuit. Optocoupler combines an LED and a photo-transistor in a single housing (package)
- ✓ In electronic circuits, optocoupler is used for suppressing interference in data communication, circuit isolation, High voltage separation, simultaneous separation and intensification signal etc
- ✓ Optocouplers can be used in either input circuits or in output circuits



Optocoupler

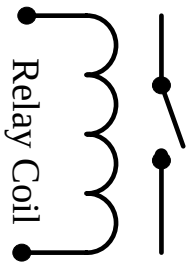


Optocoupler in input and output circuit

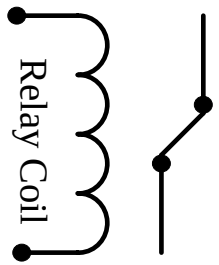
The Typical Embedded System

The I/O Subsystem – I/O Devices – Relay

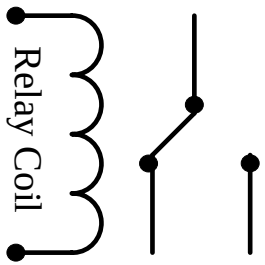
- ✓ An electro mechanical device which acts as dynamic path selectors for signals and power
- ✓ The 'Relay' unit contains a relay coil made up of insulated wire on a metal core and a metal armature with one or more contacts.
- ✓ 'Relay' works on electromagnetic principle. When a voltage is applied to the relay coil, current flows through the coil, which in turn generates a magnetic field. The magnetic field attracts the armature core and moves the contact point. The movement of the contact point changes the power/signal flow path



Single Pole Single Throw Normally Open



Single Pole Single Throw Normally Closed

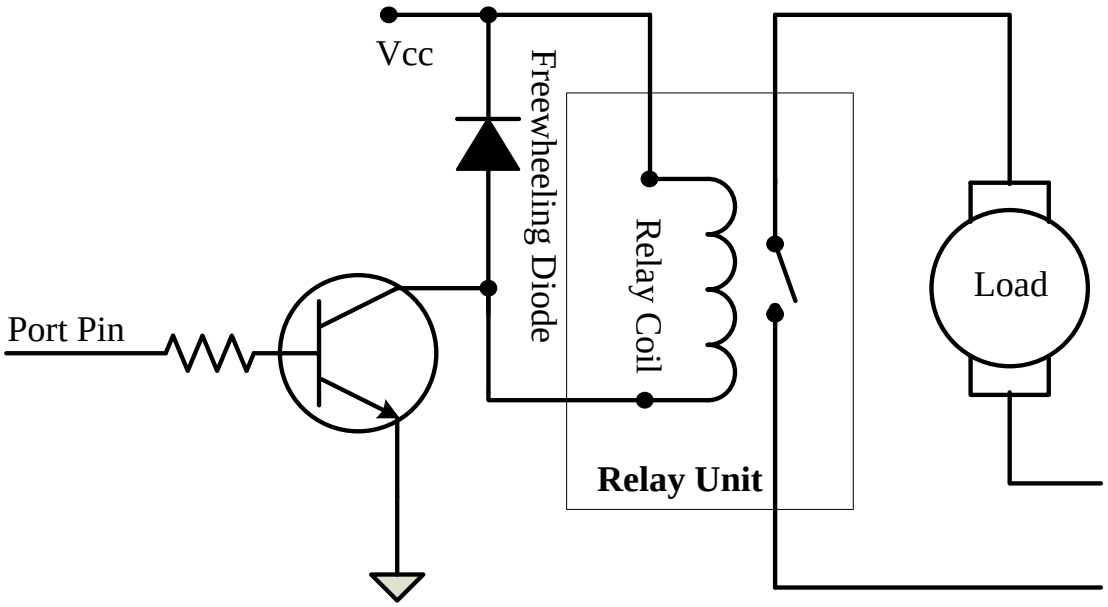


Single Pole Double Throw

The Typical Embedded System

The I/O Subsystem – I/O Devices – Relay Driver Circuit

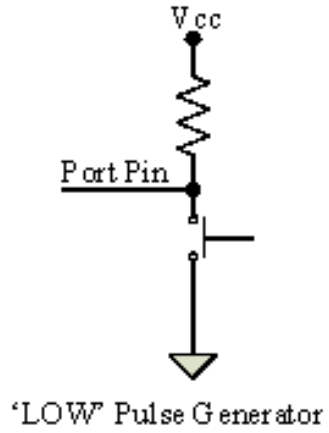
- ✓ The Relay is normally controlled using a relay driver circuit connected to the port pin of the processor/controller
- ✓ A transistor can be used as the relay driver. The transistor can be selected depending on the relay driving current requirements

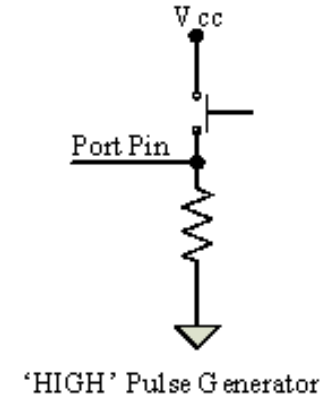


The Typical Embedded System

The I/O Subsystem – I/O Devices – Push button switch

- ✓ Push Button switch is an input device
- ✓ Push button switch comes in two configurations, namely 'Push to Make' and 'Push to Break'
- ✓ The switch is normally in the open state and it makes a circuit contact when it is pushed or pressed in the 'Push to Make' configuration
- ✓ In the 'Push to Break' configuration, the switch is normally in the closed state and it breaks the circuit contact when it is pushed or pressed
- ✓ The push button stays in the 'closed' (For Push to Make type) or 'open' (For Push to Break type) state as long as it is kept in the pushed state and it breaks/makes the circuit connection when it is released
- ✓ Push button is used for generating a momentary pulse





The Typical Embedded System

Communication Interface

- ✓ Communication interface is essential for communicating with various subsystems of the embedded system and with the external world
- ✓ For an embedded product, the communication interface can be viewed in two different perspectives; namely; Device/board level communication interface (Onboard Communication Interface) and Product level communication interface (External Communication Interface)
- ✓ Embedded product is a combination of different types of components (chips/devices) arranged on a Printed Circuit Board (PCB). The communication channel which interconnects the various components within an embedded product is referred as Device/board level communication interface (Onboard Communication Interface)
- ✓ Serial interfaces like I2C, SPI, UART, 1-Wire etc and Parallel bus interface are examples of 'Onboard Communication Interface'
- ✓ The 'Product level communication interface' (External Communication Interface) is responsible for data transfer between the embedded system and other devices or modules
- ✓ The external communication interface can be either wired media or wireless media and it can be a serial or parallel interface. Infrared (IR), Bluetooth (BT), Wireless LAN (Wi-Fi), Radio Frequency waves (RF), GPRS etc are examples for wireless communication interface
- ✓ RS-232C/RS-422/RS 485, USB, Ethernet (TCP-IP), IEEE 1394 port, Parallel port, CF-II Slot, SDIO, PCMCIA etc are examples for wired interfaces

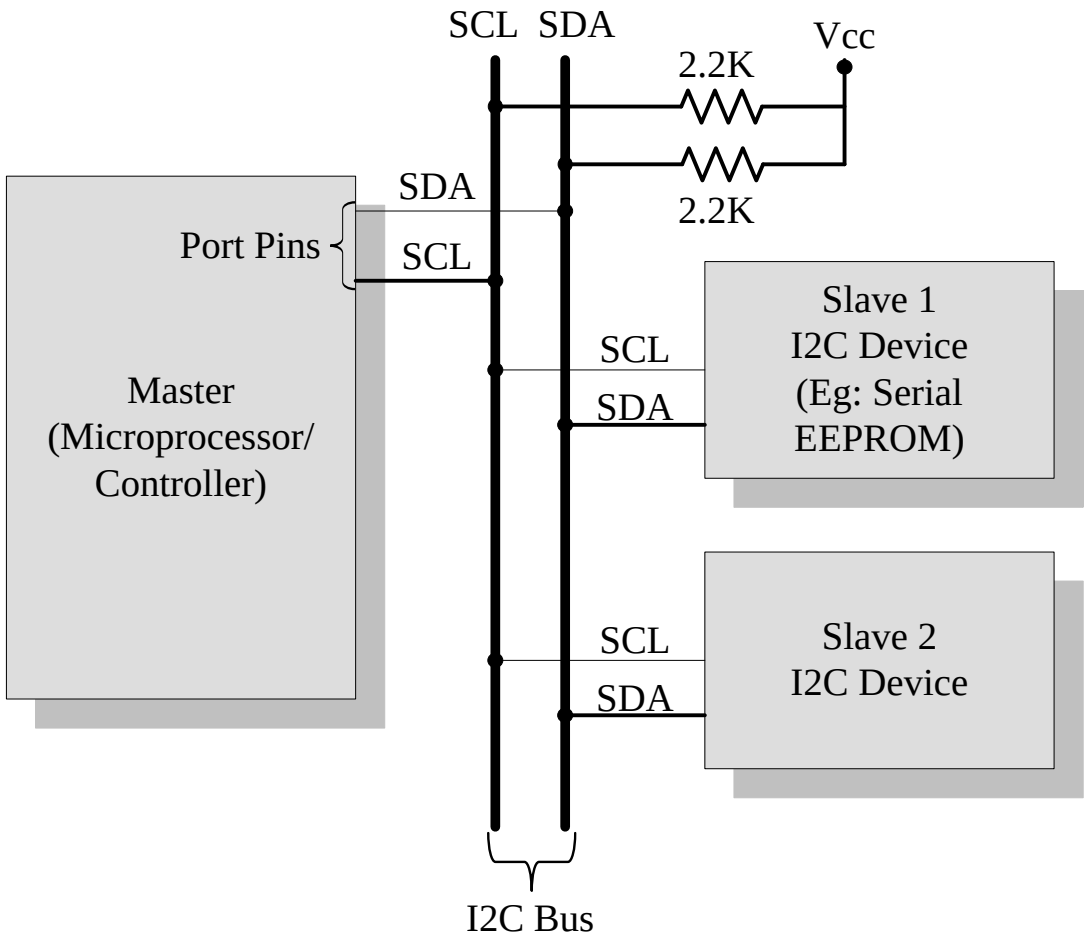
The Typical Embedded System

On-board Communication Interface - I2C

- ✓ Inter Integrated Circuit Bus (I2C - Pronounced 'I square C') is a synchronous bi-directional half duplex (one-directional communication at a given point of time) two wire serial interface bus
- ✓ The concept of I2C bus was developed by 'Philips Semiconductors' in the early 1980's. The original intention of I2C was to provide an easy way of connection between a microprocessor/microcontroller system and the peripheral chips in Television sets
- ✓ The I2C bus is comprised of two bus lines, namely; Serial Clock – SCL and Serial Data – SDA. SCL line is responsible for generating synchronization clock pulses and SDA is responsible for transmitting the serial data across devices.
- ✓ I2C bus is a shared bus system to which many number of I2C devices can be connected. Devices connected to the I2C bus can act as either 'Master' device or 'Slave' device
- ✓ The 'Master' device is responsible for controlling the communication by initiating/terminating data transfer, sending data and generating necessary synchronization clock pulses
- ✓ 'Slave' devices wait for the commands from the master and respond upon receiving the commands
- ✓ 'Master' and 'Slave' devices can act as either transmitter or receiver
- ✓ Regardless whether a master is acting as transmitter or receiver, the synchronization clock signal is generated by the 'Master' device only
- ✓ I2C supports multi masters on the same bus

The Typical Embedded System

On-board Communication Interface - I2C



The Typical Embedded System

On-board Communication Interface - I2C

The sequence of operation for communicating with an I2C slave device is:

1. Master device pulls the clock line (SCL) of the bus to 'HIGH'
2. Master device pulls the data line (SDA) 'LOW', when the SCL line is at logic 'HIGH' (This is the 'Start' condition for data transfer)
3. Master sends the address (7 bit or 10 bit wide) of the 'Slave' device to which it wants to communicate, over the SDA line. Clock pulses are generated at the SCL line for synchronizing the bit reception by the slave device. The MSB of the data is always transmitted first. The data in the bus is valid during the 'HIGH' period of the clock signal
4. Master sends the Read or Write bit (Bit value = 1 Read Operation; Bit value = 0 Write Operation) according to the requirement
5. Master waits for the acknowledgement bit from the slave device whose address is sent on the bus along with the Read/Write operation command. Slave devices connected to the bus compares the address received with the address assigned to them

5. The Slave device with the address requested by the master device responds by sending an acknowledge bit (Bit value =1) over the SDA line
6. Upon receiving the acknowledge bit, master sends the 8bit data to the slave device over SDA line, if the requested operation is ‘Write to device’. If the requested operation is ‘Read from device’, the slave device sends data to the master over the SDA line
7. Master waits for the acknowledgement bit from the device upon byte transfer complete for a write operation and sends an acknowledge bit to the slave device for a read operation
8. Master terminates the transfer by pulling the SDA line ‘HIGH’ when the clock line SCL is at logic ‘HIGH’ (Indicating the ‘STOP’ condition)

The Typical Embedded System

On-board Communication Interface – Serial Peripheral Interface (SPI) Bus

The Serial Peripheral Interface Bus (SPI) is a synchronous bi-directional full duplex four wire serial interface bus. The concept of SPI is introduced by Motorola. SPI is a single master multi-slave system. It is possible to have a system where more than one SPI device can be master, provided the condition only one master device is active at any given point of time, is satisfied. SPI requires four signal lines for communication. They are:

Master Out Slave In (MOSI): Signal line carrying the data from master to slave device. It is also known as Slave Input/Slave Data In (SI/SDI)

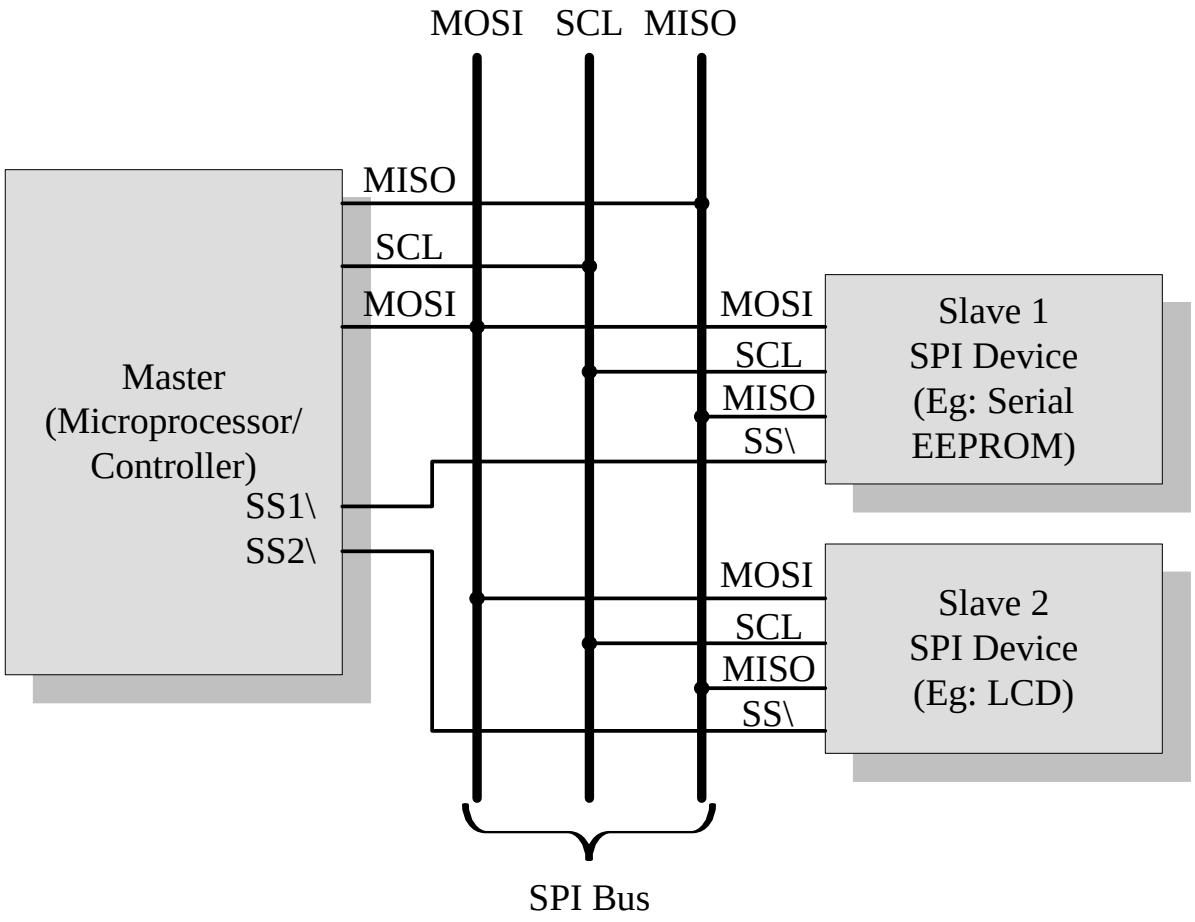
Master In Slave Out (MISO): Signal line carrying the data from slave to master device. It is also known as Slave Output (SO/SDO)

Serial Clock (SCLK): Signal line carrying the clock signals

Slave Select (SS): Signal line for slave device select. It is an active low signal

The Typical Embedded System

On-board Communication Interface – Serial Peripheral Interface (SPI) Bus



On-board Communication Interface – Serial Peripheral Interface (SPI) Bus

- ✓ The master device is responsible for generating the clock signal. Master device selects the required slave device by asserting the corresponding slave device's slave select signal 'LOW'. The data out line (MISO) of all the slave devices when not selected floats at high impedance state

- ✓ The serial data transmission through SPI Bus is fully configurable.

- ✓ SPI devices contain certain set of registers for holding these configurations.

- ✓ The Serial Peripheral Control Register holds the various configuration parameters like master/slave selection for the device, baudrate selection for communication, clock signal control etc.

- ✓ The status register holds the status of various conditions for transmission and reception.

SPI

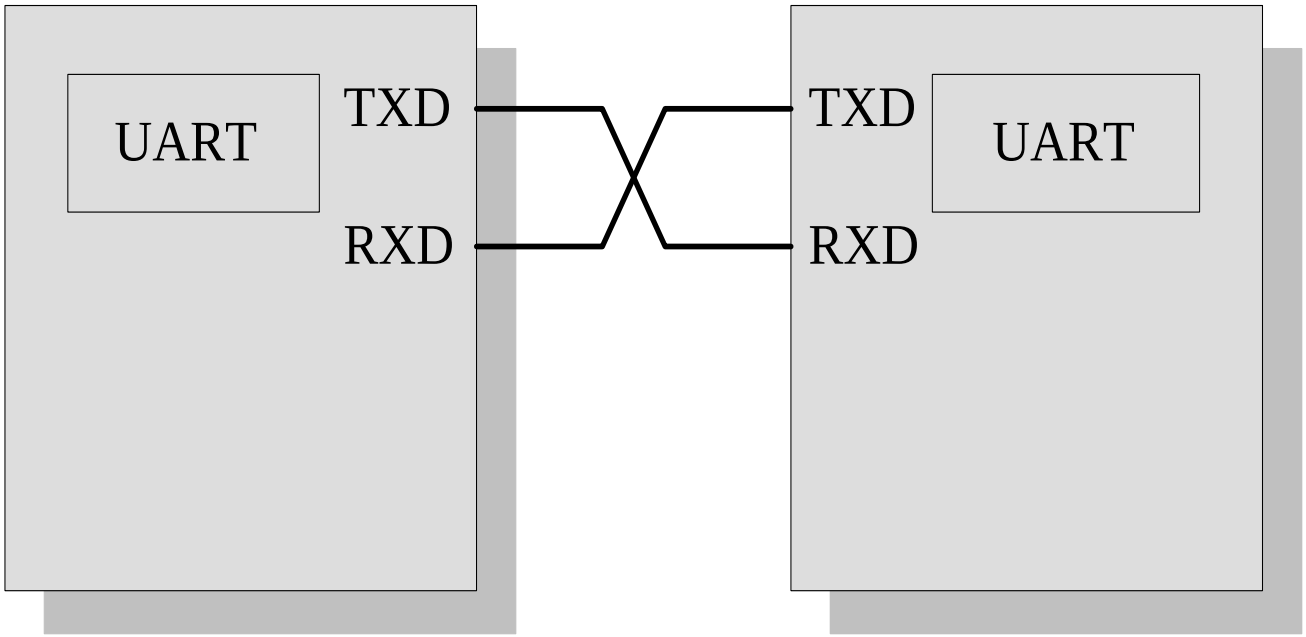
- SPI works on the principle of 'Shift Register'. The master and slave devices contain a special shift register for the data to transmit or receive.
- The size of the shift register is device dependent. Normally it is a multiple of 8.
- During transmission from the master to slave, the data in the master's shift register is shifted out to the MOSI pin and it enters the shift register of the slave device through the MOSI pin of the slave device.
- At the same time the shifted out data bit from the slave device's shift register enters the shift register of the master device through MISO pin
- Disadv – No acknowledgement mechanism.
- Adv – Can have more than one master.

On-board Communication Interface – Universal Asynchronous Receiver Transmitter (UART)

- ✓ Universal Asynchronous Receiver Transmitter (UART) based data transmission is an asynchronous form of serial data transmission
- ✓ The serial communication settings (Baudrate, No. of bits per byte, parity, No. of start bits and stop bit and flow control) for both transmitter and receiver should be set as identical
- ✓ The start and stop of communication is indicated through inserting special bits in the data stream
- ✓ While sending a byte of data, a start bit is added first and a stop bit is added at the end of the bit stream. The least significant bit of the data byte follows the start bit.
- ✓ The ‘Start’ bit informs the receiver that a data byte is about to arrive. The receiver device starts polling its ‘receive line’ as per the baudrate settings
- ✓ If parity is enabled for communication, the UART of the transmitting device adds a parity bit .
- ✓ The UART of the receiving device calculates the parity of the bits received and compares it with the received parity bit for error checking.
- ✓ The UART of the receiving device discards the ‘Start’, ‘Stop’ and ‘Parity’ bit from the received bit stream and converts the received serial bit data to a word.

The Typical Embedded System

On-board Communication Interface – Universal Asynchronous Receiver Transmitter (UART)



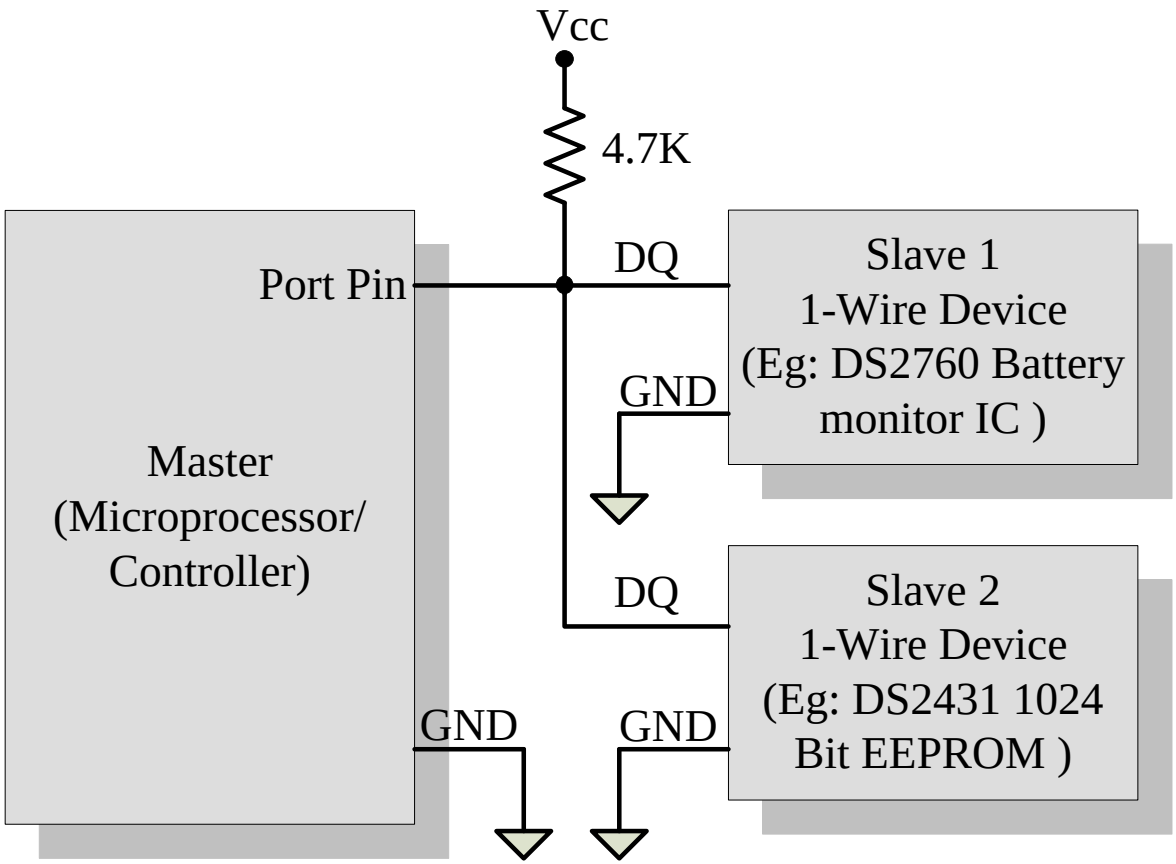
TXD: Transmitter Line
RXD: Receiver Line

On-board Communication Interface – 1-Wire Interface

- ✓ An asynchronous half-duplex communication protocol developed by Maxim Dallas Semiconductor (http://www.maxim-ic.com)
- ✓ It is also known as Dallas 1-Wire® protocol. It makes use of only a single signal line (wire) called DQ for communication and follows the master-slave communication model.
- ✓ The 1-Wire interface supports a single master and one or more slave devices on the bus
- ✓ The 1-Wire is capable of carrying power to the slave device apart from carrying the signals. Slave devices incorporate internal capacitor to generate power to operate the device from the 1-Wire
- ✓ Every 1-Wire device contains a globally unique 64 bit identification number stored within it. This unique identification number can be used for addressing individual devices present in the bus in case there are multiple slave devices connected to the 1-Wire bus
- ✓ The identifier has three parts: an 8 bit family code, a 48 bit serial number and an 8 bit CRC computed from the first 56 bits

The Typical Embedded System

On-board Communication Interface – 1-Wire Interface



On-board Communication Interface – 1-Wire Interface

The sequence of operation for communicating with a 1-Wire slave device is:

1. Master device sends a ‘Reset’ pulse on the 1-Wire bus.
2. Slave device(s) present on the bus respond with a ‘Presence’ pulse.
3. Master device sends a ROM Command (Net Address Command followed by the 64 bit address of the device). This addresses the slave device(s) to which it wants to initiate a communication
4. Master device sends a read/write function command to read/write the internal memory or register of the slave device.
5. Master initiates a Read data /Write data from the device or to the device

On-board Communication Interface – 1-Wire Interface

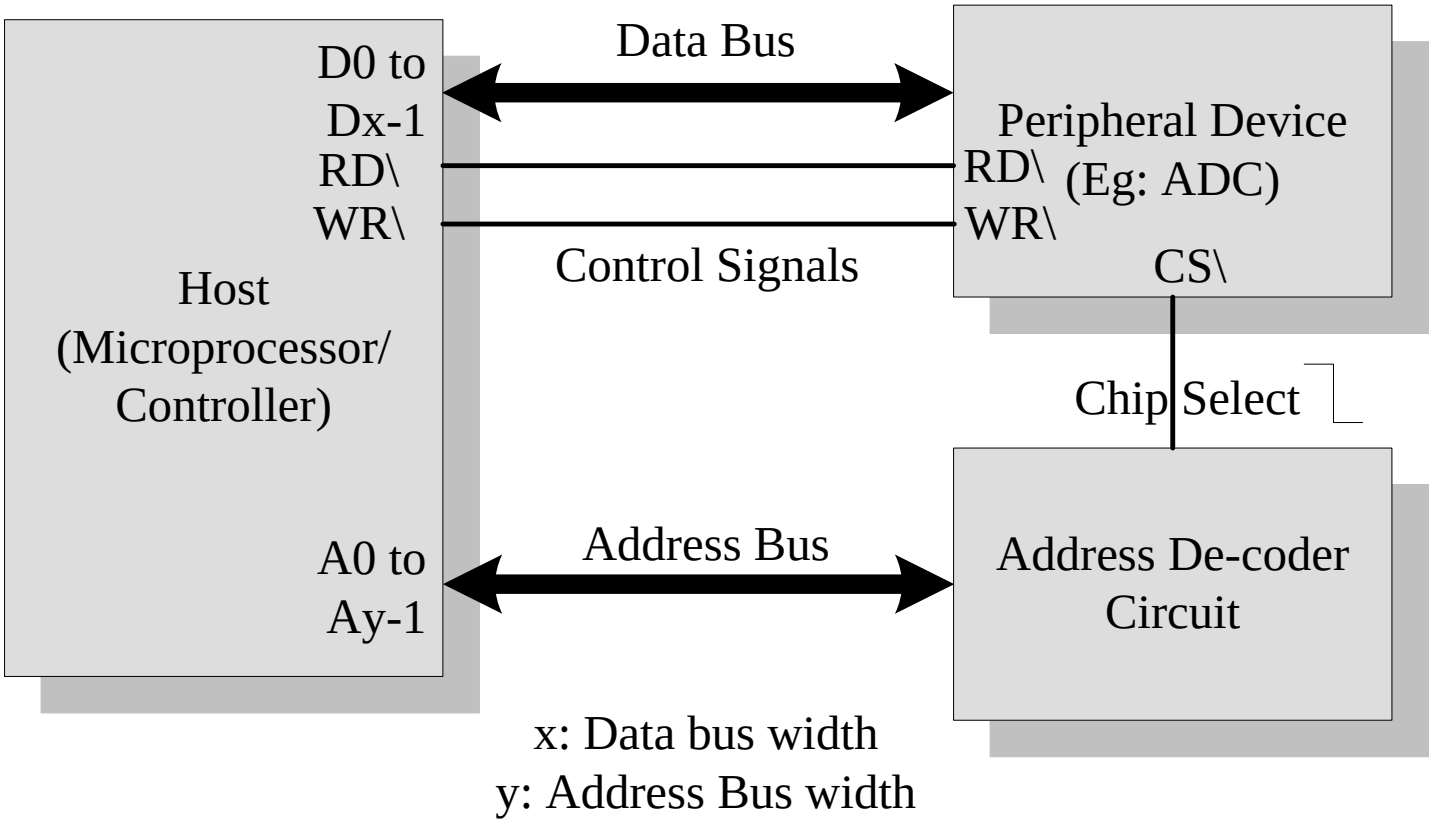
- ✓ All communication over the 1-Wire bus is master initiated
- ✓ The communication over the 1-Wire bus is divided into timeslots of 60 microseconds
- ✓ The 'Reset' pulse occupies 8 time slots. For starting a communication, the master asserts the reset pulse by pulling the 1-Wire bus 'LOW' for at least 8 time slots (480µs)
- ✓ If a 'Slave' device is present on the bus and is ready for communication it should respond to the master with a 'Presence' pulse, within 60µs of the release of the 'Reset' pulse by the master
- ✓ The slave device(s) responds with a 'Presence' pulse by pulling the 1-Wire bus 'LOW' for a minimum of 1 time slot (60µs)
- ✓ For writing a bit value of 1 on the 1-Wire bus, the bus master pulls the bus for 1 to 15µs and then releases the bus for the rest of the time slot
- ✓ A bit value of '0' is written on the bus by master pulling the bus for a minimum of 1 time slot (60µs) and a maximum of 2 time slots (120µs)
- ✓ To Read a bit from the slave device, the master pulls the bus 'LOW' for 1 to 15µs
- ✓ If the slave wants to send a bit value '1' in response to the read request from the slave, it simply releases the bus for the rest of the time slot
- ✓ If the slave wants to send a bit value '0', it pulls the bus 'LOW' for the rest of the time slot.

On-board Communication Interface – Parallel Interface

- ✓ Parallel interface is normally used for communicating with peripheral devices which are memory mapped to the host of the system.
- ✓ The host processor/controller of the embedded system contains a parallel bus and the device which supports parallel bus can directly connect to this bus system
- ✓ The communication through the parallel bus is controlled by the control signal interface between the device and the host.
- ✓ The ‘Control Signals’ for communication includes ‘Read/Write’ signal and device select signal.
- ✓ The device normally contains a device select line and the device becomes active only when this line is asserted by the host processor.
- ✓ The direction of data transfer (Host to Device or Device to Host) can be controlled through the control signal lines for ‘Read’ and ‘Write’. Only the host processor has control over the ‘Read’ and ‘Write’ control signals

The Typical Embedded System

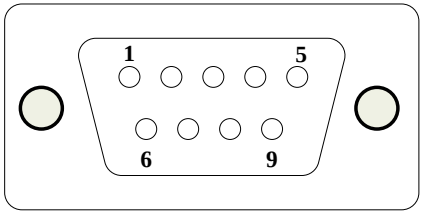
On-board Communication Interface – Parallel Interface



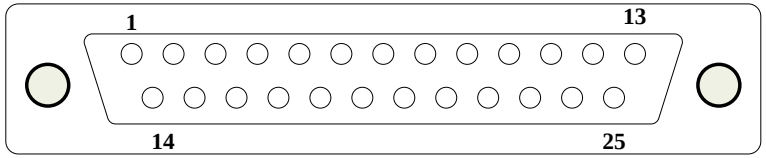
The Typical Embedded System

External Communication Interface – RS-232 C & RS-485

- ✓ RS-232 C (Recommended Standard number 232, revision C from the **Electronic Industry Association**) is a legacy, full duplex, wired, asynchronous serial communication interface
- ✓ RS-232 extends the UART communication signals for external data communication.
- ✓ UART uses the standard TTL/CMOS logic (Logic 'High' corresponds to bit value 1 and Logic 'LOW' corresponds to bit value 0) for bit transmission whereas RS232 use the EIA standard for bit transmission. As per EIA standard, a logic '0' is represented with voltage between +3 and +25V and a logic '1' is represented with voltage between -3 and -25V. In EIA standard, logic '0' is known as 'Space' and logic '1' as 'Mark'.
- ✓ The RS232 interface define various handshaking and control signals for communication apart from the 'Transmit' and 'Receive' signal lines for data communication. RS-232 supports two different types of connectors, namely; DB-9: 9-Pin connector and DB-25: 25-Pin connector.



DB-9



DB-25

The Typical Embedded System

External Communication Interface – RS-232 C & RS-485

Pin Name	Pin No: (For DB-9 Connector)	Description
TXD	3	Transmit Pin. Used for Transmitting Serial Data
RXD	2	Receive Pin. Used for Receiving serial Data
RTS	7	Request to send.
CTS	8	Clear To Send
DSR	6	Data Set ready
GND	5	Signal Ground
DCD	1	Data Carrier Detect
DTR	4	Data Terminal Ready
RI	9	Ring Indicator

External Communication Interface – RS-232 C & RS-485

- ✓ RS-232 is a point-to-point communication interface and the devices involved in RS-232 communication are called 'Data Terminal Equipment (DTE)' and 'Data Communication Equipment (DCE)'

- ✓ If no data flow control is required, only TXD and RXD signal lines and ground line (GND) are required for data transmission and reception. The RXD pin of DCE should be connected to the TXD pin of DTE and vice versa for proper data transmission.

- ✓ If hardware data flow control is required for serial transmission, various control signal lines of the RS-232 connection are used appropriately. The control signals are implemented mainly for modem communication and some of them may be irrelevant for other type of devices.

- ✓ The Request To Send (RTS) and Clear To Send (CTS) signals co-ordinate the communication between DTE and DCE. Whenever the DTE has a data to send, it activates the RTS line and if the DCE is ready to accept the data, it activates the CTS line

RS-232

- ✓ The Data Terminal Ready (DTR) signal is activated by DTE when it is ready to accept data. The Data Set Ready (DSR) is activated by DCE when it is ready for establishing a communication link. DTR should be in the activated state before the activation of DSR.

- ✓ The Data Carrier Detect (DCD) is used by the DCE to indicate the DTE that a good signal is being received

- ✓ Ring Indicator (RI) is a modem specific signal line for indicating an incoming call on the telephone line

External Communication Interface – RS-232 C & RS-485

- ✓ As per the EIA standard RS-232 C supports baudrates up to 20Kbps (Upper limit 19.2Kbps) The commonly used baudrates by devices are 300bps, 1200bps, 2400bps, 9600bps, 11.52Kbps and 19.2Kbps.

- ✓ The maximum operating distance supported in RS-232 communication is 50 feet at the highest supported baudrate.

- ✓ Embedded devices contain a UART for serial communication and they generate signal levels conforming to TTL/CMOS logic.

- ✓ A level translator IC like MAX 232 from Maxim Dallas semiconductor is used for converting the signal lines from the UART to RS-232 signal lines for communication.

- ✓ On the receiving side the received data is converted back to digital logic level by a converter IC. Converter chips contain converters for both transmitter and receiver..

- ✓ RS-232 uses single ended data transfer and supports only point-to-point communication and not suitable for multi-drop communication.

- ✓ RS-422 is another serial interface standard from EIA for differential data communication. It supports data rates up to 100Kbps and distance up to 400 ft.

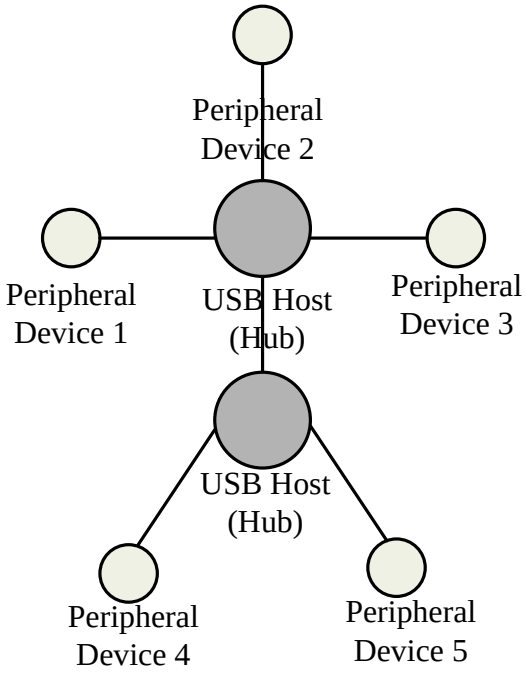
- ✓ RS-422 supports multi-drop communication with one transmitter device and receiver devices up to 10.

- ✓ RS-485 is the enhanced version of RS-422 and it supports multi-drop communication with up to 32 transmitting devices (drivers) and 32 receiving devices on the bus.

- ✓ The communication between devices in the bus makes use of the ‘addressing’ mechanism to identify slave devices

External Communication Interface – Universal Serial Bus (USB)

- ✓ Universal Serial Bus (USB) is a wired high speed serial bus for data communication
- ✓ The USB communication system follows a star topology with a USB host at the center and one or more USB peripheral devices/USB hosts connected to it
- ✓ A USB host can support connections up to 127, including slave peripheral devices and other USB hosts
- ✓ USB transmits data in packet format. Each data packet has a standard format. The USB communication is a host initiated one
- ✓ The USB Host contains a host controller which is responsible for controlling the data communication, including establishing connectivity with USB slave devices, packetizing and formatting the data packet.
- ✓ There are different standards for implementing the USB Host Control interface; namely Open Host Control Interface (OHCI) and Universal Host Control Interface (UHCI)



The Typical Embedded System

External Communication Interface – Universal Serial Bus (USB)

- ✓ The Physical connection between a USB peripheral device and master device is established with a USB cable
- ✓ The USB cable supports communication distance of up to 5 meters
- ✓ The USB standard uses two different types of connectors namely ‘Type A’ and ‘Type B’ at the ends of the USB cable for connecting the USB peripheral device and host device
- ✓ ‘Type A’ connector is used for upstream connection (connection with host) and ‘Type B’ connector is used for downstream connection (connection with slave device)

Pin No:	Pin Name	Description
1	V_{BUS}	Carries power (5V)
2	D-	Differential data carrier line
3	D+	Differential data carrier line
4	GND	Ground signal line

External Communication Interface – Universal Serial Bus (USB)

- ✓ Each USB device contains a Product ID (PID) and a Vendor ID (VID)
- ✓ The PID and VID are embedded into the USB chip by the USB device manufacturer
- ✓ The VID for a device is supplied by the USB standards forum.
- ✓ PID and VID are essential for loading the drivers corresponding to a USB device for communication.
- ✓ USB supports four different types of data transfers, namely; Control, Bulk, Isochronous and Interrupt.
- ✓ Control transfer is used by USB system software to query, configure and issue commands to the USB device

- ✓ Bulk transfer is used for sending a block of data to a device. Bulk transfer supports error checking and correction. Transferring data to a printer is an example for bulk transfer.

- ✓ Isochronous data transfer is used for real time data communication. In Isochronous transfer, data is transmitted as streams in real time. Isochronous transfer doesn't support error checking and re-transmission of data in case of any transmission loss

- ✓ Interrupt transfer is used for transferring small amount of data. Interrupt transfer mechanism makes use of polling technique to see whether the USB device has any data to send

- ✓ The frequency of polling is determined by the USB device and it varies from 1 to 255 milliseconds. Devices like Mouse and Keyboard, which transmits fewer amounts of data, uses Interrupt transfer.

External Communication Interface – IEEE 1394 (Firewire)

- ✓ A wired, isochronous high speed serial communication bus. It is also known as High Performance Serial Bus (HPSB)
- ✓ The research on 1394 was started by Apple Inc in 1985 and the standard for this was coined by IEEE.
- ✓ The Apple Inc's (www.apple.com) implementation of 1394 protocol is popularly known as ***Firewire***.
- ✓ ***i.LINK*** is the 1394 implementation from Sony Corporation (www.sony.net) and ***Lynx*** is the implementation from Texas Instruments (www.ti.com)
- ✓ 1394 supports peer-to-peer connection and point-to-multipoint communication allowing 63 devices to be connected on the bus in a tree topology
- ✓ The 1394 standard supports a data rate of 400 to 3200Mbits/Second
- ✓ IEEE 1394 uses differential data transfer and the interface cable supports 3 types of connectors, namely; 4-pin connector, 6-pin connector (alpha connector) and 9 pin connector (beta connector)
- ✓ The 6 and 9 pin connectors carry power also to support external devices. It can supply unregulated power in the range of 24 to 30V (The Apple implementation is for battery operated devices and it can supply a voltage in the range 9 to 12V)

The Typical Embedded System

External Communication Interface – IEEE 1394 (Firewire)

Pin Name	Pin No: (4 Pin Connector)	Pin No: (6 Pin Connector)	Pin No: (9 Pin Connector)	Description
Power		1	8	Unregulated DC supply. 24 to 30V
Signal Ground		2	6	Ground connection
TPB-	1	3	1	Differential Signal line for Signal Line B
TPB+	2	4	2	Differential Signal line for Signal Line B
TPA-	3	5	3	Differential Signal line for Signal Line A
TPA+	4	6	4	Differential Signal line for Signal Line A
TPA(S)			5	Shield for the differential signal line A. Normally grounded
TPB(S)			9	Shield for the differential signal line B. Normally grounded
NC			7	No connection

The Typical Embedded System

External Communication Interface – IEEE 1394 (Firewire)

- ✓ The IEEE 1394 connector contains two differential data transfer lines namely A and B
- ✓ The differential lines of A are connected to B (TPA+ to TPB+ and TPA- to TPB-) and vice versa
- ✓ Unlike USB interface (Except USB OTG), IEEE 1394 doesn't require a host for communicating between devices. Example, a scanner can be directly connected to a printer for printing
- ✓ The data rate supported by 1394 is far higher than the one supported by USB2.0 interface
- ✓ 1394 is a popular communication interface for connecting embedded devices like 'Digital Camera', 'Camcorder', 'Scanners' with desktop Computers for data transfer and storage

External Communication Interface – Infrared (IrDA)

- ✓ A serial, half duplex, line of sight based wireless technology for data communication between devices.
- ✓ Infrared communication technique makes use of Infrared waves of the electromagnetic spectrum for transmitting the data
- ✓ IrDA supports point-point and point-to-multipoint communication, provided all devices involved in the communication are within the line of sight
- ✓ The typical communication range for IrDA lies in the range 10cm to 1 m

- ✓ IR supports data rates ranging from 9600bits/second to 16Mbps.
- ✓ Depending on the speed of data transmission IR is classified into Serial IR (SIR), Medium IR (MIR), Fast IR (FIR), Very Fast IR (VFIR) and Ultra Fast IR (UFIR).
- ✓ SIR supports transmission rates ranging from 9600bps to 115.2kbps. MIR supports data rates of 0.576Mbps and 1.152Mbps. FIR supports data rates up to 4Mbps. VFIR is designed to support high data rates up to 16Mbps. The UFIR specs are under development and it is targeting a data rate up to 100Mbps
- ✓ IrDA communication involves a transmitter unit for transmitting the data over IR and a receiver for receiving the data. Infrared Light Emitting Diode (LED) is used as the IR source for transmitter and at the receiving end a photodiode is used as the receiver

External Communication Interface – Bluetooth

- ✓ Low cost, low power, short range wireless technology for data and voice communication.
- ✓ Bluetooth operates at 2.4GHz of the Radio Frequency spectrum and uses the Frequency Hopping Spread Spectrum (FHSS) technique for communication.
- ✓ Bluetooth supports a theoretical maximum data rate of up to 1Mbps and a range of approximately 30 feet for data communication
- ✓ Bluetooth communication has two essential parts; a physical link part and a protocol part. The physical link is responsible for the physical transmission of data between devices supporting Bluetooth communication and protocol part is responsible for defining the rules of communication.
- ✓ The physical link works on the Wireless principle making use of RF waves for communication

- ✓ Bluetooth enabled devices essentially contain a Bluetooth wireless radio for the transmission and reception of data.

- ✓ The rules governing the Bluetooth communication is implemented in the 'Bluetooth protocol stack'. The Bluetooth communication IC holds the stack.

- ✓ Each Bluetooth device will have a 48 bit unique identification number. Bluetooth communication follows packet based data transfer.

- ✓ Bluetooth supports point-to-point (device to device) and point-to-multipoint (device to multiple device broadcasting) wireless communication. The point-to-point communication follows the master-slave relationship. A Bluetooth device can function as either master or slave.

- ✓ A network formed with one Bluetooth device as master and more than one device as slaves is known as Piconet

External Communication Interface – Wi-Fi

- ✓ The popular wireless communication technique for networked communication of devices.
- ✓ Wi-Fi follows the IEEE 802.11 standard.
- ✓ Wi-Fi is intended for network communication and it supports Internet Protocol (IP) based communication.
- ✓ Wi-Fi based communications require an intermediate agent called Wi-Fi router/Wireless Access point to manage the communications.
- ✓ The Wi-Fi router is responsible for restricting the access to a network, assigning IP address to devices on the network, routing data packets to the intended devices on the network.

- ✓ Wi-Fi enabled devices contain a wireless adaptor for transmitting and receiving data in the form of radio signals through an antenna.

- ✓ Wi-Fi operates at 2.4GHZ or 5GHZ of radio spectrum and they co-exist with other ISM band devices like Bluetooth.

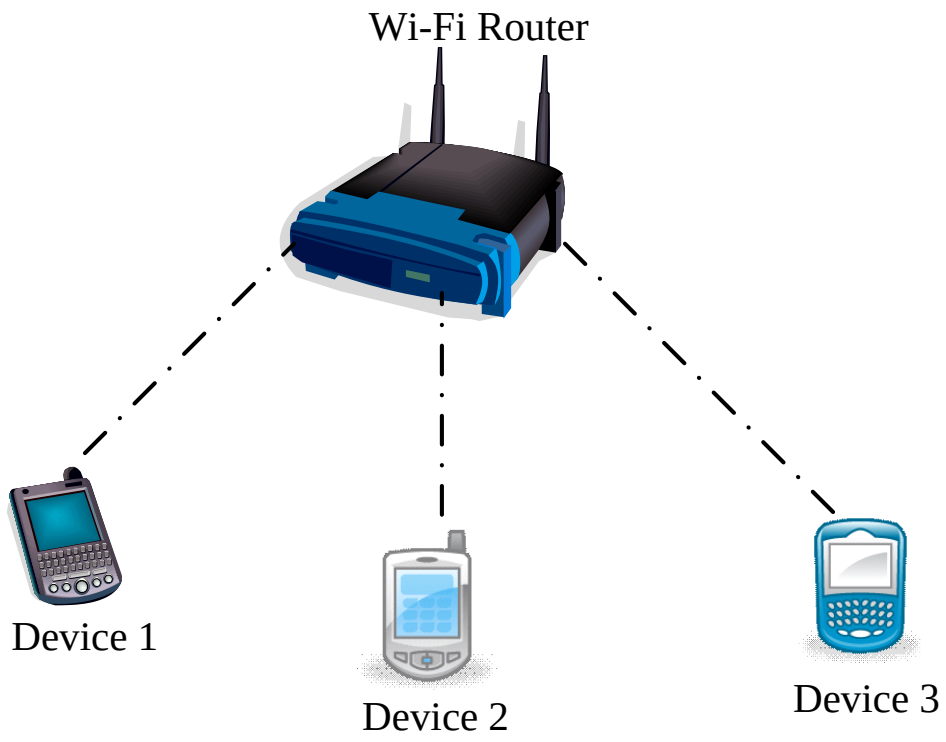
- ✓ A Wi-Fi network is identified with a Service Set Identifier (SSID). A Wi-Fi device can connect to a network by selecting the SSID of the network and by providing the credentials if the network is security enabled.

- ✓ Wi-Fi networks implements different security mechanisms for authentication and data transfer.

- ✓ Wireless Equivalency Protocol (WEP), Wireless Protected Access (WPA) etc are some of the security mechanisms supported by Wi-Fi networks in data communication

The Typical Embedded System

External Communication Interface – Wi-Fi



The Typical Embedded System

External Communication Interface – ZigBee

- ✓ Low power, low cost, wireless network communication protocol based on the IEEE 802.15.4-2006 standard.
- ✓ ZigBee is targeted for low power, low data rate and secure applications for Wireless Personal Area Networking (WPAN).
- ✓ The ZigBee specifications support a robust mesh network containing multiple nodes. This networking strategy makes the network reliable by permitting messages to travel through a number of different paths to get from one node to another.
- ✓ ZigBee operates worldwide at the unlicensed bands of Radio spectrum, mainly at 2.400 to 2.484 GHz, 902 to 928 MHz and 868.0 to 868.6 MHz

- ✓ ZigBee Supports an operating distance of up to 100 meters and a data rate of 20 to 250Kbps.

- ✓ ZigBee is primarily targeting application areas like Home & Industrial Automation, Energy Management, Home control/security, Medical/Patient tracking, Logistics & Asset tracking and sensor networks & active RFID.

- ✓ Automatic Meter Reading (AMR), smoke and detectors, wireless telemetry, HVAC control, heating control, Lighting controls, Environmental controls, etc are examples for applications which can make use of the ZigBee technology

The Typical Embedded System

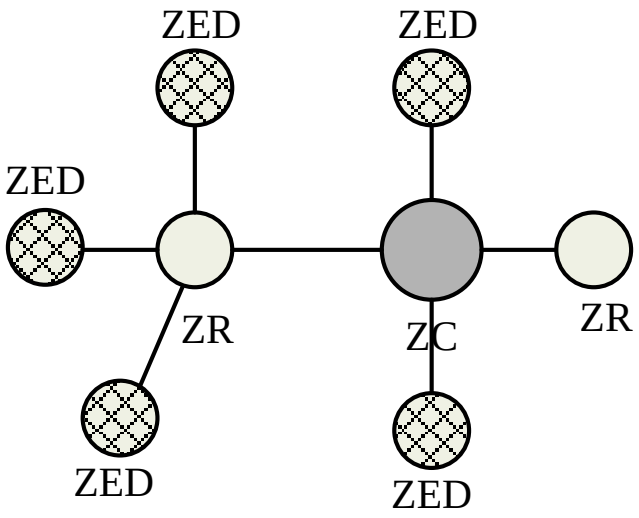
External Communication Interface – ZigBee

In the ZigBee terminology, each ZigBee device falls under any one of the following ZigBee device category

ZigBee Coordinator (ZC)/Network Coordinator: The ZigBee coordinator acts as the root of the ZigBee network. The ZC is responsible for initiating the ZigBee network and it has the capability to store information about the network

ZigBee Router (ZR)/Full function Device (FFD): Responsible for passing information from device to another device or to another ZR

ZigBee End Device (ZED)/Reduced Function Device (RFD): End device containing ZigBee functionality for data communication. It can talk only with a ZR or ZC and doesn't have the capability to act as a mediator for transferring data from one device to another.



The Typical Embedded System

External Communication Interface – General Packet Radio Service (GPRS)

- ✓ A communication technique for transferring data over a mobile communication network like GSM
- ✓ Data is sent as packets. The transmitting device splits the data into several related packets. At the receiving end the data is re-constructed by combining the received data packets
- ✓ GPRS supports a theoretical maximum transfer rate of 171.2kbps
- ✓ In GPRS communication, the radio channel is concurrently shared between several users instead of dedicating a radio channel to a cell phone user. The GPRS communication divides the channel into 8 timeslots and transmits data over the available channel
- ✓ GPRS supports Internet Protocol (IP), Point to Point Protocol (PPP) and X.25 protocols for communication.
- ✓ GPRS is mainly used by mobile enabled embedded devices for data communication. The device should support the necessary GPRS hardware like GPRS modem and GPRS radio
- ✓ GPRS is an old technology and it is being replaced by new generation data communication techniques like EDGE, High Speed Downlink Packet Access (HSDPA) etc which offers higher bandwidths for communication

The Typical Embedded System

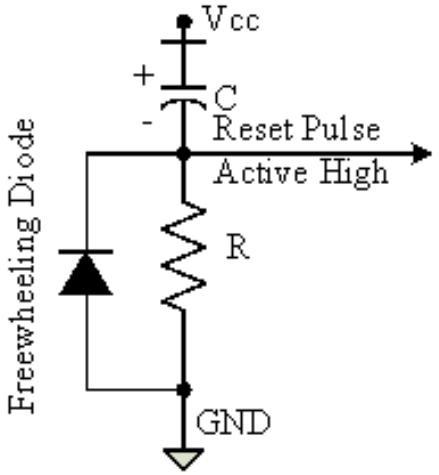
Embedded Firmware

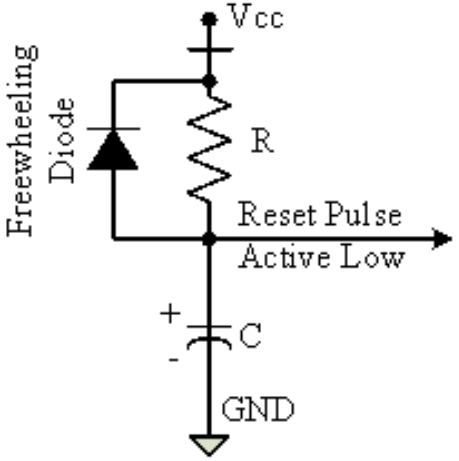
- ✓ The control algorithm (Program instructions) and or the configuration settings that an embedded system developer dumps into the code (Program) memory of the embedded system
- ✓ The embedded firmware can be developed in various methods like
  - ✓ Write the program in high level languages like Embedded C/C++ using an Integrated Development Environment (The IDE will contain an editor, compiler, linker, debugger, simulator etc. IDEs are different for different family of processors/controllers.
  - ✓ Write the program in Assembly Language using the Instructions Supported by your application's target processor/controller

The Typical Embedded System

Other System Components – Reset Circuit

- ✓ The Reset circuit is essential to ensure that the device is not operating at a voltage level where the device is not guaranteed to operate, during system power ON
- ✓ The Reset signal brings the internal registers and the different hardware systems of the processor/controller to a known state and starts the firmware execution from the reset vector (Normally from vector address 0x0000 for conventional processors/controllers
- ✓ The reset vector can be relocated to an address for processors/controllers supporting bootloader
- ✓ The reset signal can be either active high (The processor undergoes reset when the reset pin of the processor is at logic high) or active low (The processor undergoes reset when the reset pin of the processor is at logic low).

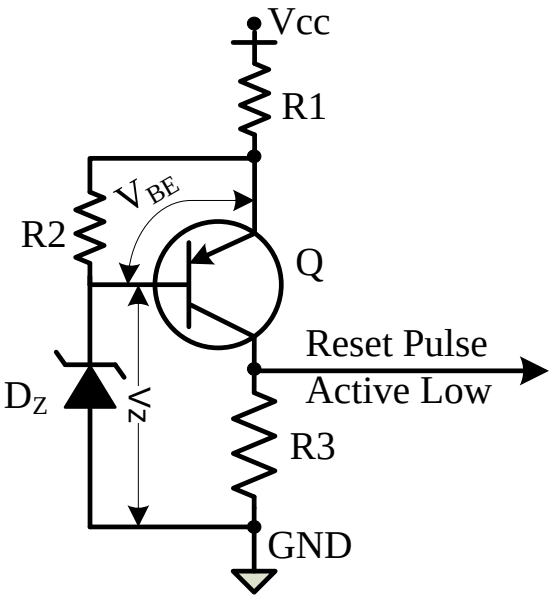




The Typical Embedded System

Other System Components – Brown-out Protection Circuit

- ✓ Brown-out protection circuit prevents the processor/controller from unexpected program execution behavior when the supply voltage to the processor/controller falls below a specified voltage
- ✓ The processor behavior may not be predictable if the supply voltage falls below the recommended operating voltage. It may lead to situations like data corruption
- ✓ A brown-out protection circuit holds the processor/controller in reset state, when the operating voltage falls below the threshold, until it rises above the threshold voltage
- ✓ Certain processors/controllers support built in brown-out protection circuit which monitors the supply voltage internally
- ✓ If the processor/controller doesn't integrate a built-in brown-out protection circuit, the same can be implemented using external passive circuits or supervisor ICs

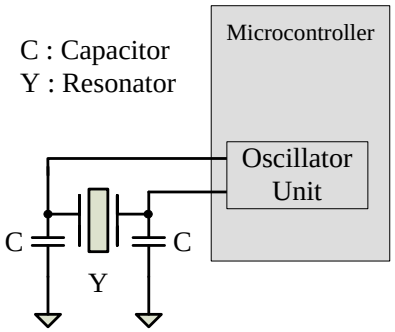


The Typical Embedded System

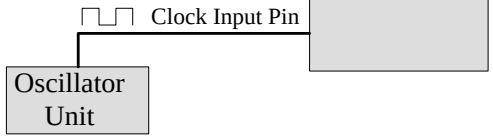
Other System Components – Oscillator Unit

- ✓ A microprocessor/microcontroller is a digital device made up of digital combinational and sequential circuits
- ✓ The instruction execution of a microprocessor/controller occurs in sync with a clock signal
- ✓ The oscillator unit of the embedded system is responsible for generating the precise clock for the processor
- ✓ Certain processors/controllers integrate a built-in oscillator unit and simply require an external ceramic resonator/quartz crystal for producing the necessary clock signals
- ✓ Certain processor/controller chips may not contain a built-in oscillator unit and require the clock pulses to be generated and supplied externally
- ✓ Quartz crystal Oscillators are example for clock pulse generating devices

Quartz Crystal Resonator



Crystal Oscillator



The Typical Embedded System

Other System Components – Real Time Clock (RTC)

- ✓ The system component responsible for keeping track of time. RTC holds information like current time (In hour, minutes and seconds) in 12 hour /24 hour format, date, month, year, day of the week etc and supplies timing reference to the system
- ✓ RTC is intended to function even in the absence of power. RTCs are available in the form of Integrated Circuits from different semiconductor manufacturers like Maxim/Dallas, ST Microelectronics etc
- ✓ The RTC chip contains a microchip for holding the time and date related information and backup battery cell for functioning in the absence of power, in a single IC package
- ✓ The RTC chip is interfaced to the processor or controller of the embedded system
- ✓ For Operating System based embedded devices, a timing reference is essential for synchronizing the operations of the OS kernel. The RTC can interrupt the OS kernel by asserting the interrupt line of the processor/controller to which the RTC interrupt line is connected
- ✓ The OS kernel identifies the interrupt in terms of the Interrupt Request (IRQ) number generated by an interrupt controller
- ✓ One IRQ can be assigned to the RTC interrupt and the kernel can perform necessary operations like system date time updation, managing software timers etc when an RTC timer tick interrupt occurs

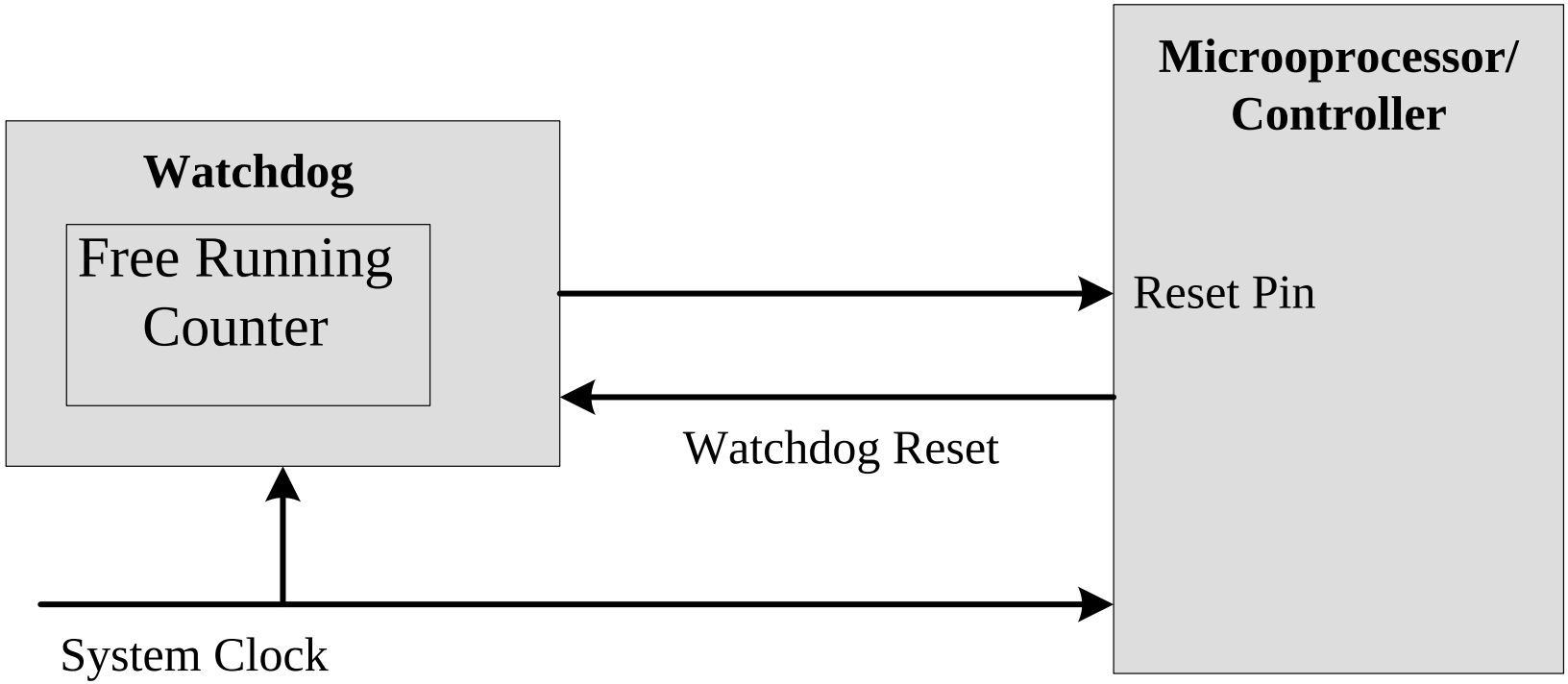
The Typical Embedded System

Other System Components – Watch Dog Timer (WDT)

- ✓ A timer unit for monitoring the firmware execution
- ✓ Depending on the internal implementation, the watchdog timer increments or decrements a free running counter with each clock pulse and generates a reset signal to reset the processor if the count reaches zero for a down counting watchdog, or the highest count value for an up counting watchdog
- ✓ If the watchdog counter is in the enabled state, the firmware can write a zero (for up counting watchdog implementation) to it before starting the execution of a piece of code (subroutine or portion of code which is susceptible to execution hang up) and the watchdog will start counting. If the firmware execution doesn't complete due to malfunctioning, within the time required by the watchdog to reach the maximum count, the counter will generate a reset pulse and this will reset the processor
- ✓ If the firmware execution completes before the expiration of the watchdog timer the WDT can be stopped from action
- ✓ Most of the processors implement watchdog as a built-in component and provides status register to control the watchdog timer (like enabling and disabling watchdog functioning) and watchdog timer register for writing the count value. If the processor/controller doesn't contain a built in watchdog timer, the same can be implemented using an external watchdog timer IC circuit.

The Typical Embedded System

Other System Components – Watch Dog Timer (WDT)



External Watch Dog Timer Unit Interfacing with Processor

WDT - Applications

- An application in mobile phone is that display is off in case no GUI interaction takes place within a watched time interval.
- The interval is usually set at 15 s, 20 s, 25 s, 30 s in mobile phone.
- This saves power.

Thank you